$J_m||C_{max}$: Modelli ed Approcci Risolutivi

Davide Brauzi, Università di Roma Tor Vergata, Dipartimento di Ingegneria dell'Impresa

Sommario—In questo documento viene presentato nel dettaglio il **J**ob **S**hop **S**cheduling **P**roblem (JSSP) con obiettivo minimizzazione del **tempo di completamento** (Makespan), noto in letteratura come $J_m||C_{max}$. Anzitutto (Sezione (1)), viene definito il problema di Scheduling in ambiente Job Shop su un numero generico di macchinari, con una breve trattazione dello Stato dell'Arte (Sezione (2)). Vengono presentate tre Formulazioni Matematiche (Sezione (3)), esplicitando variabili, vincoli e parametri tramite **M**ixed **I**nteger **P**rogram (MIP); questi modelli vengono poi confrontati su istanze di piccole dimensioni vista la grande complessità del problema in esame. Viene quindi descritta l'euristica *Shifting Bottleneck* (Sezione (4)), supportando l'analisi con risultati sperimentali (Sezione (4.6)).
I risultati presentati sono stati ottenuti su varie istanze implementate e risolte in **AMPL** (**A** **M**athematical **P**rogramming **L**anguage). Le procedure risolutive e i concetti teorici utilizzati sono tratti dalle lezioni del corso di *Modelli per la Gestione di Sistemi complessi*, tenuto presso l'Università di Roma Tor Vergata dal prof. Stefano Giordani.

Index Terms—JSSP, Disgiuntive Model, Shifting Bottleneck, Industrial Optimization, Makespan.

1 Definizione del problema

Presentiamo il Problema di Job Shop Scheduling con obiettivo minimizzazione del Makespan (1, Giordani); sia dato un ambiente di processamento composto un insieme $M = \{M_1, \dots, M_m\}$ di macchine dedicate (ciascuna macchina può eseguire un certo insieme di lavorazioni Φ_i). Sia $J = J_1, J_2, \dots, J_n$ l'insieme dei job da processare, ciascuno costituito da un insieme di attività $\Omega_j = \{T_j^1, T_j^2, \dots, T_j^h, \dots, T_j^{n_j}\}$. Ogni attività deve essere eseguita su una precisa macchina $\mu_j^h \in M$, ed esistono vincoli di precedenza (noti) tra le attività di un job j, secondo cui l'attività h-esima del generico job j deve precedere direttamente l'attività (h+1)-esima:

$$Prec(T_j^h, T_j^{h+1}) \triangleq T_j^h < T_j^{h+1} \quad h = 1 \dots n_j - 1 \qquad (1)$$

Questo equivale a dire che le rotte dei vari job tra le macchine sono note a priori ed esulano dal processo decisionale. Si consideri inoltre l'impossibilità di interrompere un'attività in esecuzione su un macchinario e si assuma che un job visiti un macchinario al più una volta.
Tale assunzione, consente la seguente definizione:

$$T_{i,j} \triangleq T_j^h : \mu_j^h = M_i \quad i \in M, j \in J$$
$$\phi_i \triangleq \{T_{i,j}\} \quad i \in M, j \in J$$

Si associa dunque a ciascuna attività un tempo di processamento $p_{j,h}$ (deterministico): senza perdita di generalità sia $p_{j,h} \in \mathbb{N}$. In questo modo si ha la possibilità di considerare sempre istanze discrete.
Si consideri inoltre che ciascun macchinario possa eseguire al più un'operazione alla volta (vincolo di non sovarapposizione tra attività che competono per la stessa risorsa), e che esso disponga di un buffer in uscita di capacità (modellisticamente) illimitata in modo da non generare fenomeni di **blockage**.
Supponiamo infatti, che il job j abbia completato il suo processamento sul macchinario M_i all'istante τ, che $T_{i,j} < T_{l,j}$ e che non vi siano buffer all'uscita; supponiamo inoltre che M_l non sia libera all'istante τ. In questa situazione M_i non potrebbe essere liberata, originando i tipici fenomeni di attesa dei casi reali.
Con un buffer di capacità pari al numero di operazioni da eseguire sul macchinario M_i ($|\Phi_i|$), questo fenomeno può essere evitato.

Ricordiamo infine che uno stesso job non può essere processato contemporaneamente su due risorse differenti (vincolo di non sovrapposizione tra attività di uno stesso job).
All'interno di questo sistema (che può a ben vedere essere considerato complesso), l'obiettivo è schedulare i job sui vari macchinari in modo da minimizzare il tempo di completamento del job che temina più tardi.
Nella tassonomia $\alpha|\beta|\gamma$ introdotta da (2, Graham et al.), questo problema viene indicato con $J_m||C_{max}$.
Di seguito rappresentiamo la rotta del generico job j:

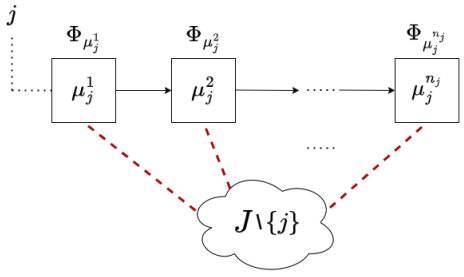


Figura 1: Rotta job e conflitti

La figura riesce a descrivere chiaramente il processo decisionale: risolvere in maniera ottima (rispetto al makespan) i conflitti sui vari macchinari rispettando le rotte dei job su di essi. Vedremo che questa interpretazione ci porterà a riformulare il problema mediante modello reticolare.

1.1 Rappresentazione su calcolatore

Concludiamo questa sezione introduttiva, definendo il metodo di rappresentazione sintetica delle istanze su calcolato-

re.
Un'istanza è tipicamente descritta da una coppia di interi (n, m) e da una matrice $P \in \mathbb{N}^{nx(2g)}$, $g = \max_j\{n_j\}$, così definita:

$$\begin{cases} P(j, 2h-1) = \mu_j^h \\ P(j, 2h) = p_{\mu_j^h j} \end{cases} \qquad j = 1, .., n \; ; h = 1..g$$

Ogni riga della matrice è dunque relativa ad un job, e la h-esima ipercolonna (formata dalla coppia di elementi sopra riportata) rappresenta la macchina occupata dall'operazione h job j e il relativo tempo di processamento.
Senza perdita di generalità, se un job j non transita per il macchinario i si considera $p_{ij} = 0$ in modo da avere una rappresentazione coerente senza modificare l'istanza di interesse.

2 Stato dell'Arte

Il problema introdotto è sicuramente uno dei problemi di ottimizzazione combinatoria più studiati in ambito accademico, per tre motivi:

1) Applicabilità: molti ambienti produttivi e logistici sono modellabili come Job-Shop.
2) Possibilità di Estensione: il modello $J_m||Cmax$ è il più semplice modello di Job-Shop Scheduling; le ipotesi semplificative di questo modello, infatti, possono essere rilassate in modi diversi ((3, Xiong et Al.)). Un esempio tipico in letteratura è il **DJSSP** (Dynamic Job Shop Scheduling Problem), nel quale una o più delle seguenti ipotesi vengono eliminate per maggiore aderenza con i casi reali:
    a) Tutti i job sono pronti all'istante di inizio pianificazione e il numero di job è noto a priori.
    b) I job non presentano *due date*.
    c) I tempi di processamento sono univocamente determinati.
3) Complessità computazionale.

Uno dei primi articoli in letteratura contenenti il termine *Job-Shop* è quello di Sisson (4), in cui si definisce un ambiente produttivo caratterizzato dal fatto che la sequenza di operazioni (rotta) seguita da un lotto (job) è indipendente dalla sequenza di operazioni richiesta da qualsiasi altro lotto (job).

I ricercatori operativi e gli studiosi della matematica applicata si sono da subito scontrati con la elevata complessità del problema; in (5, Garey et Al.) viene per la prima volta dimostrata l'NP-completezza di questo problema per $m \geq 3$, tramite riduzione del *3-Partition* a $F_3||Cmax$ (un caso particolare di $J_3||Cmax$).

Questo fatto ha fatto crescere l'attenzione nei confronti del problema, tramite lo sviluppo di metodi esatti ed euristici per la sua risoluzione. Partiamo con analizzare gli approcci esatti:

- *Branch-and-bound*: il più classico schema di risoluzione è quello di (6, Giffler e Thompson), in cui si generano tutte le schedule attive basandosi su un sottoinsieme di attività schedulabili che viene dinamicamente aggiornato. Basandosi invece sulla rappresentazione del problema mediante *Grafo Disgiuntivo* (descritta nella Sezione (3.1)), sono stati sviluppati metodi di Branch and Bound, sempre più efficienti grazie a sofisticati LB che consentono un'enumerazione più snella [(7, Carlier e Pinson), (8, Barker e Mchmahon)].
- *Formulazione Matematica*: le formulazioni matematiche per il problema in esame si differenziano in base a diverse interpretazioni modellistiche; esempi diffusi sono il Disgiuntive Graph Model (DM) tratto da [(9, Balas),(10, Roy e Sussman)] e basato sulla rappresentazione reticolare del problema e il Time Indexed Model (TI) tratto dal ben più generale problema di Project Scheduling con risorse limitate affrontato da (11, Pritsker et Al.) e basato sull'assegnazione dell'inizio (fine) delle attività a precisi istanti di tempo. Ricordiamo inoltre il Rank Based Model (RB) tratto da (12, Wagner) e basato sulla definizione delle sequenze sui vari macchinari mediante approccio posizionale (attività diverse occupano posizioni diverse).
- Programmazione Dinamica: metodi basati sulla formulazione di (13, Held) per i problemi di sequenziamento non sono molto diffusi a causa delle dimensione proibitiva dello spazio di stato. I riferimenti accessibili sono piuttosto scarsi, ma sembrerebbe ragionevole che lo *spazio di stato* includa:
    - Attività completate.
    - Tempo corrente su ciascun macchinario.

    Per quanto riguarda invece lo *spazio di azione*, esso potrebbe essere composto dalle attività schedulabili sui macchinari (dove per attività schedulabile si intende un'attività il cui predecessore diretto è stato schedulato).

Relativamente ai metodi non esatti, già in (14, Lawler et Al.) si fa riferimento a numerose euristiche; ricordiamo per esempio la performante *Shifting Bottleneck* (15, Adams et Al.) (che analizzeremo nel dettaglio nella Sezione (4)) e le varie procedure di *Simulated Annealing* [(16, Matsuo), (17, Van Laarhoven et Al.)].
Gli algoritmi di Simulated Annealing (strutturati su Processi decisionali Markoviani) si sono naturalmente evoluti nelle più moderne tecniche di Reinforcement Learning, aumentando la propria efficienza ed efficacia grazie ai diffusi incrementi della potenza di calcolo. Citiamo in particolare il lavoro di (18, Zhang), che sfrutta tecniche di *Deep Reinforcement Learning* per apprendere la politica di schedulazione delle attività, tramite episodi caratterizzati da configurazioni successive (e coerenti) del Grafo Disgiuntivo.
Sono diffuse inoltre tecniche di *Tabu Search* (TS), nelle quali si evita la convergenza in minimi locali, accettando transizioni peggiorative ed evitando di visitare soluzioni già esplorate per un dato numero di operazioni (l'algoritmo ha "memoria a breve termine"); tra le prime applicazioni della TS a $J_m||C_{max}$ ricordiamo (19, Taillard). Si sono poi sviluppati numerosi miglioramenti di questa prima architettura algoritmica, ragionando in particolare sul concetto

di *Vicinato* (intorno): un insieme di soluzioni ammissibili ottenuto applicando piccole (e computazionalmente poco onerose) perturbazioni alla soluzione corrente. Si è dimostrata la maggiore efficacia di vicinati ottenuti perturbando la struttura dei *blocchi critici*: sequenze massimali di operazioni critiche processate sullo stesso macchinario. In (20, Zhang) vengono illustrati alcuni metodi di generazione di vicinati noti in letteratura, descrivendo poi una nuova procedura di TS basata sull' interscambio di operazioni di uno stesso blocco critico.

Non volendo presentare una trattazione esaustiva riguardo tutti gli approcci modellistici e risolutivi, è mia premura concludere questa sezione con una riflessione di carattere generale. $J_m||C_{max}$ è uno dei problemi combinatori più complessi studiati in letteratura, e come tale risulta piuttosto delicato; la sua risoluzione con un singolo metodo è a mio parere insufficiente per ottenere buoni risultati in tempi ragionevoli, e per questo preferisco pensare a metodi ibridi, che possano integrare più strategie sequenzialmente. L'idea che più mi sembra interessante, e che al dire il vero è tutt'altro che nuova, è quella di utilizzare tecniche di (Deep) Reinforcement Learning, o euristiche combinatorie, per generare velocemente soluzioni di buona qualità, da utilizzare eventualmente come *hot-start* per algoritmi di Branch and Bound progettati ad-hoc (con il fine ovviamente di rendere più efficiente l'enumerazione implicita).

3 Formulazione matematica

In questa sezione descriviamo 3 formulazioni MIP per il problema trattato:

1) **Disgiuntive Graph Model**: basato sulla rappresentazione reticolare del problema.
2) **Time Indexed Model**: basato sull'indicizzazione temporale degli *starting time* delle attività di ciascun job.
3) **Rank Based Model**: modello posizionale.

Analizzeremo poi le loro prestazioni e la loro complessità computazionale, ragionando sul numero di variabili e vincoli che le compongono (Sezione 3.4).

3.1 Disgiuntive Graph Model (DM)

Prima di introdurre la formulazione matematica del problema, descriviamo la sua rappresentazione reticolare classica [(10, Roy e Sussman) (9, Balas)]. In questo modello si rappresenta l'ambiente produttivo su un grafo disgiuntivo definito come segue:

$$G = (V, E, \{A_1 \cup A_2\}),$$
$$\begin{cases} V \triangleq \{s,t\} \cup \{T_{j,h}\} \quad j = 1,\dots,n \quad h = 1,\dots,n_j \\ E \triangleq \{(T_j^h\,,\,T_p^l),(T_p^l\,,\,T_j^h)\} \quad \forall (T_j^h\,,\,T_p^l) : \mu_j^h = \mu_p^l \\ A_1 \triangleq \{(T_j^h\,,\,T_j^{h+1})\} \quad j = 1,\dots,n \quad h = 1,\dots,n_j - 1 \\ A_2 \triangleq \{(s\,,\,T_j^1),(T_j^{n_j}\,,\,t)\} \quad j = 1,\dots,n \end{cases} \tag{2}$$

V rappresenta l'insieme dei nodi del grafo, identificato con tutte le attività dei job. I nodi $\{s,t\}$, vengono aggiunti per fornire un pozzo ed una sorgente in modo da adattare la rappresentazione modellistica a vari algoritmi risolutivi (come vedremo nella Sezione 4).
In questa rappresentazione si definiscono due tipi di relazione binarie:

- **Precedenze dirette**: tramite gli **archi congiuntivi** A_1 ed A_2; in particolare, A_1 rappresenta le precedenze esistenti tra le attività di un singolo job (catena). A_2, invece, rappresenta la precedenza del nodo (attività) sorgente s rispetto alla prima attività di ciascun job, e la precedenza di ogni ultima attività di ciascun job rispetto al nodo (attività) pozzo.
- **Conflitti sui macchinari**: tramite gli **archi disgiuntivi** E, rappresentanti l'impossibilità di eseguire contemporaneamente due attività che competono per lo stesso macchinario (notiamo che tali archi non sono orientati). In questo senso si hanno m **clique** di attività.

Il grafo risulta parametrizzato ed è dunque correttamente definibile come **rete di flusso**; i parametri in questione sono i pesi sui nodi $p_j^h \in \mathbb{N},\ p_s = p_t = 0$.
Oss: per la descrizione e la risoluzione della formulazione si possono omettere i nodi $\{s,t\}$ e, ovviamente, gli archi A_2.
Questa rappresentazione è nota in letteratura come rappresentazione **AON** (**A**ctivity **O**n **N**odes), e si contrappone alla **AOA** (**A**ctivity **O**n **A**rcs).

In questo senso una soluzione ammissibile (schedula) risulta in un orientamento aciclico degli archi disgiuntivi, che equivale a definire le sequenze sui macchinari $\sigma_i\ , i = 1,\dots,m$. Prima di riportare la formulazione che consente di identificare la schedula ottima, riportiamo rappresentazioni qualitative di alcuni elementi del grafo disgiuntivo:

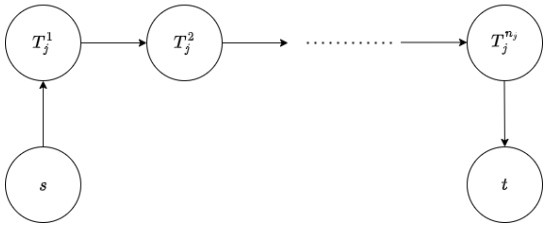


Figura 2: Catena di precedenze su grafo disgiuntivo

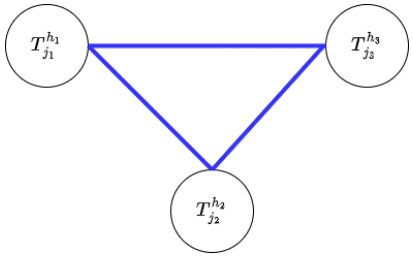


Figura 3: Clique generata da archi disgiuntivi

A questo punto introduciamo le variabili del problema

$$x^i_{jh,pl} := \begin{cases} 1 & \text{se} \quad T^h_j < T^l_p \\ 0 & \text{altrimenti} \end{cases}$$
$$i \in M, j = 1 \dots n-1, h \in \Omega_j : \mu^h_j = i$$
$$p = j+1 \dots n, l \in \Omega_p : \mu^l_p = i$$

$s_{j,h} \geq 0$ starting time attivita' h del job j.
$j \in J, \quad h \in \Omega_j$

$C_{max} \geq 0$ Makespan della schedula

Per motivi di notazione definiamo il campo di variabilità della quintupla di indici (i, j, h, p, l) come $\alpha^i_{jh,pl}$, rendendo così le scritture più compatte.

$$\min \quad C_{\max} + \epsilon \sum_{j \in J} \sum_{h \in \Omega_j} s_{j,h}$$
$$\text{s.t. } s_{j,n_j} + p_{j,n_j} \leq C_{\max} \quad \forall j \in J \qquad (v^{DM}_1)$$
$$s_{j,h} + p_{j,h} \leq s_{j,h+1} \quad \forall j \in J,\ h = 1 \dots n_j - 1 \qquad (v^{DM}_2)$$
$$s_{j,h} + p_{j,h} \leq s_{p,l} + F(1 - x^i_{jh,pl}) \quad \forall \alpha^i_{jh,pl} \qquad (v^{DM}_3)$$
$$s_{j,h} + p_{j,h} \leq s_{p,l} + F(x^i_{jh,pl}) \quad \forall \alpha^i_{jh,pl} \qquad (v^{DM}_4)$$

In questo modello, dunque, vengono gestite le precedenze definite dagli archi congiuntivi tramite la variabile $s_{j,h}$ e il vincolo v^{DM}_2, mentre i conflitti sui macchinari (risorse) sono gestiti mediante la variabile $x^i_{jh,pl}$ e la coppia di vincoli $v^{DM}_3 - v^{DM}_4$. Proprio questi vincoli infatti funzionano in *mutua esclusione* tramite l'impiego di un parametro $F = \sum_{j \in J} \sum_{h \in \Omega_j} p_{j,h}$ (UB al valore del tempo di completamento della schedula, ottenuto supponendo di schedulare le attività una dopo l'altra).
Resta da indagare il vincolo v^{DM}_1, relativo alla definizione stessa del Makespan. Data la struttura reticolare introdotta in Figura (2), il tempo di completamento del progetto può essere identificato con lo *starting-time* dell'attività fittizia t. Questa attività, legata da vincoli di precedenza rispetto all'ultima attività di ogni job, potrà iniziare solo dopo il completamento del processamento del job che termina più tardi; come di consueto un vincolo non lineare $C_{max} = \max_{j \in J}\{s_{j,h} + p_{j,h}\}$ viene riscritto tramite $|J|$ vincoli lineari (v^{DM}_1).
Nella f.o. compare oltre al $C - max$, il termine $\epsilon \sum_{j \in J} \sum_{h \in \Omega_j} s_{j,h}$ (dove ϵ è un valore piccolo a piacere), che consente di trovare tra tutte le schedule ottime rispetto al Makespan, quelle *Earliest-start*.

Concludiamo la descrizione del modello con una tabella relativa al numero di vincoli e variabili della formulazione in esame, considerando senza perdita di generalità istanze in cui $n_j = m, \forall j \in J$:

	v^{DM}_1	v^{DM}_2	$v^{DM}_3 - v^{DM}_4$	TOT
NCONST	n	$n(m-1)$	$2m\binom{n}{2}$	$O(2mn^2)$

Tabella 1: Numero vincoli Disgiuntive Graph Model Formulation (DM)

	$x^i_{jh,pl}$	$s_{j,h}$	$C_{\max}$	TOT
NVAR	$\binom{n}{2}m$	$n \cdot m$	1	$O(mn^2)$

Tabella 2: Numero variabili Disgiuntive Graph Model Formulation (DM)

3.2 Time Indexed Model (TI)

Questo modello è articolato su una differente rappresentazione modellistica dell'ambiente di processamento, basata sulla rappresentazione dell'asse temporale sul quale schedulare le diverse attività. Adattiamo la proposta di (11, Pritsker et Al.) relativa al più generale **RCPSP** (**R**esource **C**onstrained **P**roject **S**cheduling **P**roblem) al problema in esame. Consideriamo un Upper Bound T al valore del Makespan ottimo (per esempio quello dato da $\sum_{j \in J} \sum_{h \in \Omega_j} p_{j,h}$) e suddividiamo l'intervallo temporale $[0, T]$ in T sottointervalli di dimensione unitaria; di seguito forniamo una rappresentazione grafica di quanto appena detto per maggiore chiarezza:

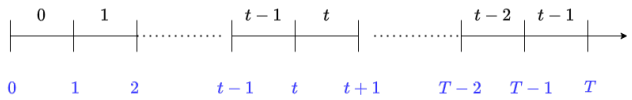


Figura 4: Asse tempi TI Formulation

Come è possibile notare, al generico sottointervallo elementare $(t, t+1]$ è assegnato l'indice t, $t = 0 \dots T-1$.
Questa rappresentazione consente in maniera semplice di assegnare gli *starting-time* di ciascuna attività $T_{j,h}$ a precisi intervalli elementari, data la natura discreta dei loro *processing-time*. Introduciamo pertanto le seguenti variabili definendo preliminarmente $H \triangleq \{0 \dots T-1\}$:

$$x^t_{j,h} := \begin{cases} 1 & \text{se l'attivita' } T^h_j \text{ inizia all'istante } t \\ 0 & \text{altrimenti} \end{cases}$$
$$j \in J,\ h \in \Omega_j,\ t = 0 \dots T - p_{j,h}.$$

$C_{max} \geq 0$ Makespan della schedula

Si noti la dipendenza esistente tra il campo di variabilità dell'indice t e l'UB fornito sul Makespan ottimo T; questo fatto ci porterà a fare interessanti riflessioni quando quantificheremo NVAR per questa formulazione. Si noti che la quantità $[x^t_{j,h} t]$, rappresenta lo starting time dell'attività $T_{j,h}$ se e solo se $x^t_{j,h} = 1$. Prima di riportare la formulazione definiamo anche in questo caso in modo compatto il campo di variabilità dell'indice t come $\gamma_{j,h}(1)$; si riporta un (1) in quanto, come sarà più chiaro a breve, è possibile definire il campo di variabilità di t in maniera più efficiente. Fatta

questa premessa, è lecito scrivere quanto segue:

$$\min \quad C_{\max} + \epsilon \sum_{j\in J} \sum_{h\in\Omega_j} \sum_{t\in\gamma^t_{j,h}(1)} x^t_{j,h} t$$

$$\text{s.t.} \sum_{t\in\gamma(1)_{j,h}} x^t_{j,h} = 1 \quad \forall j \in J,\ h \in \Omega_j \qquad (v_1^{TI})$$

$$\sum_{t\in\gamma_{j,h}(1)} x^t_{j,n_j} t + p_{j,n_j} \le C_{max} \quad \forall j \in J \qquad (v_2^{TI})$$

$$\sum_{t\in\gamma_{j,h}(1)} x^t_{j,h} t + p_{j,h} \le \sum_{t\in\gamma_{j,h+1}(1)} x^t_{j,h+1} t$$
$$\forall j \in J,\ h \in \Omega_j \backslash \{n_j\} \qquad (v_3^{TI})$$

$$\sum_{j\in J} \sum_{[h\in\Omega_j : \mu^h_j = i]} \sum_{[\theta = max(0, t-p_{j,h}+1)]}^{[min(t, T-p_{j,h})]} x^\theta_{j,h} \le 1$$
$$\forall i \in M,\ t \in H \qquad (v_4^{TI})$$

Notiamo subito che, anche in questo caso, si ricercano solo le schedule *Earliest-start* tramite il fattore correttivo $\epsilon \sum_{j\in J} \sum_{h\in\Omega_j} \sum_{t\in\gamma^t_{j,h}(1)} x^t_{j,h} t$.

Per quanto riguarda i vincoli li descriviamo brevemente: v_1^{TI} stabilisce che ogni attività deve essere schedulata (assegno un intervallo per lo starting time), v_2^{TI} è il solito vincolo per la definizione del Makespan e v_3^{TI} è il vincolo di precedenza sulle attività di un singolo job.
In questo caso risulta delicata la gestione dei conflitti sui macchinari. Coerentemente al modello introdotto, dato un generico intervallo t, e la generica macchina i al più una tra le attività $[T_{i,j} \equiv T_{j,h} : \mu^h_j = i]$ può essere in esecuzione; i casi etsremi possibili sono i seguenti:

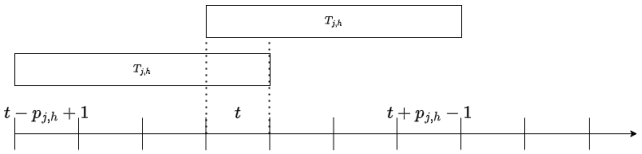


Figura 5: Processamento in intervallo t, situazioni estreme

Al limite, data la sua durata $p_{j,h}$, l'attività $T_{j,h}$ sarà in esecuzione nell'intervallo t se inizia a $t - p_{j,h} + 1$ ovvero se inizia proprio all'istante t. Ovviamente queste situazioni devono essere racchiuse all'interno del campo di definizione di t: $\gamma_{j,h}(1)$. L'intersezione di questi due intervalli è proprio data dall'intervallo:

$$[t - p_{j,h} + 1, t] \cap [0, T - p_{j,h}] =$$
$$= [max(0, t - p_{j,h} + 1), min(t, T - pj, h)],$$

si noti in particolare la coincidenza con l'esntensione della sommatoria nel vincolo v_4^{TI}.

Si può però definire in maniera più efficiente (compatta) il campo di variabilità dell'indice t nella variabile $x^t_{j,h}$. Infatti, la generica attività $T_{j,h}$, a causa dei vincoli di precedenza dati dalla rotta del job j può partire al più presto a $r^1_{j,h} \triangleq \sum_{l=1}^{h-1} p_{j,h}$ e (rispettando T) non può finire più tardi di $r^2_{j,h} \triangleq T - \sum_{l=h}^{n_j} p_{j,h}$. In questo modo possiamo definire $\gamma_{j,h}(2) \triangleq [r^1_{j,h}, r^2_{j,h}] \subseteq [0, T - p_{j,h}]$, in modo da diminuire il numero di variabili del problema. Ci riferiremo alla formulazione indicizzata su $\gamma_{j,h}(1)$ con TI(1), mentre a quella indicizzata su $\gamma_{j,h}(2)$ con TI(2).
Questa variazione lascia invariato il numero di vincoli del problema, rendendo il numero di variabili sensibilmente minore. Nonostante la riduzione del numero di variabili sia fortemente legata all'istanza risolta, giustifichiamo formalmente quanto appena asserito:

$$\sum_{j\in J} \sum_{h\in\Omega_j} r^2_{j,h} - r^1_{j,h} + 1 \le \sum_{j\in J} \sum_{h\in\Omega_j} T - p_{j,h} + 1$$
$$r^2_{j,h} \le T - p_{j,h} \quad \text{per definizione di} \quad r^2_{j,h},$$
$$r^2_{j,h} - r^1_{j,h} + 1 \le T - p_{j,h} - r^1_{j,h} + 1 \le$$
$$\le T - p_{j,h} + 1.$$

Sommando membro a membro il primo e l'ultimo termine della catena di disuguaglianze riportata si ottiene la tesi.

Prima di concludere riportando il numero di vincoli e di variabili per la formulazione in questione, vogliamo evidenziare la dipendenza del numero di variabili (e dunque della complessità risolutiva) dal valore T. Considerando infatti la formulazione TI(1), il numero di variabili $x^t_{j,h}$ può essere sottostimato come segue:

$$NVAR = \sum_{j\in J} \sum_{h\in\Omega_j} T - p_{j,h} + 1 \ge \sum_{j\in J} \sum_{h\in\Omega_j} T - p_{j,h} \ge$$
$$\ge \sum_{j\in J} \sum_{h\in\Omega_j} T - \overline{p} = nm(T - \overline{p}).$$
$$\overline{p} \triangleq \max\{p_{j,h}\}$$

Per ottenere tale identità si suppone anche in questo caso di considerare (per semplicità ma senza perdita di generalità) istanze con $n_j = m,\ \forall j \in J$.
Ora supponendo a ben vedere $T >> \overline{p}$, si può considerare NVAR $\sim O(nmT)$ il che conferma la stretta dipendenza del problema dalle dimensioni dell'istanza risolta, ovvero dalla qualità dell'UB fornito per il calcolo di T. Evidenzieremo quanto detto con opportuni risultati sperimentali nella sottosezione (3.4).

Come premesso concludiamo anche in questo caso la trattazione riportando le tabelle riassuntive per la sola formulazione TI(1) (vista l'impossibilità esprimere in forma chiusa l'ordine di grandezza di NVAR per TI(2)).

	v_1^{TI}	v_2^{TI}	v_3^{TI}	v_4^{TI}	TOT
NCONST	nm	n	$n(m-1)$	mT	$O(m(n+T))$

Tabella 3: Numero vincoli Time Indexed Formulation (TI(1))

	$x^t_{j,h}$	$C_{\max}$	TOT
NVAR	$nm(T - \overline{p})$	1	$O(mnT)$

Tabella 4: Numero variabili Time Indexed Formulation (TI(1))

3.3 Rank Based Formulation (RB)

Analizziamo ora la formulazione Rank-Based, la cui paternità viene attribuita a (12, Wagner). Il modello è basato sulle sequenze dei singoli macchinari σ_i, e sull'identificazione della posizione k-esima di queste sequenze con una specifica attività (rispettando ovviamente le rotte dei singoli job).
Moltissimi problemi di scheduling sono modellati secondo l'approccio *rank-based*, che risulta particolarmente flessibile e talvolta intuitivo.
Sarà ben presto evidente, però, che questo modello sembra essere ben più complesso (e pertanto meno efficiente) del più diffuso e adatto (DM).

Anche in questo caso rappresentiamo l'intuizione modellistica graficamente:

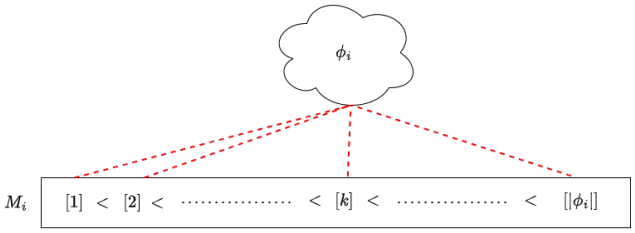


Figura 6: Assegnazione dei job che competono su un macchinario ad una specifica posizione della sua sequenza

Nella figura $[k]$ indica l' attività in posizione k-esima del generico macchinario M_i.
Come detto, in una soluzione ammissibile, le attività $T_{i,j}$ che competono per la macchina M_i, devono essere opportunamente sequenziate perchè non sovrapponibili. Il sequenziamento di tali attività si espleta mediante la realizzazione di vincoli di precedenza tra le attività stesse, come evidenziato nella figura.

Fatte le dovute premesse, definiamo anche in questo caso le variabili del problema:

$$x^i_{jh,k} := \begin{cases} 1 \text{ se } T^h_j \text{ è schedulata in posizione } [k] \text{ su } M_i \\ 0 \quad \text{altrimenti} \end{cases}$$
$$i \in M, j \in J, h \in \Omega_j : \mu^h_j = i, k = 1 \dots |\phi_i|$$

$f^i_k \geq 0$ finish-time attività in pos. k della macch. i.
$j \in J, \quad i \in M$

$C_{max} \geq 0$ Makespan della schedula

Per completezza e senza perdita di generalità, in questo caso utilizzeremo come variabile il *finishing-time* delle attività; ricordiamo infatti che *starting-time* e *finishing-time* sono variabili equivalenti, poichè legate dalla relazione:

$$f_{j,h} = s_{j,h} + p_{j,h} \tag{3}$$

Nella seguente formulazione useremo nuovamente il parametro F per indicare un buon UB al Makespan ottimo del problema:

$$\min \quad C_{\max} + \epsilon \sum_{j \in J} \sum_{k=1}^{|\phi_i|} f^i_k$$

$$\text{s.t.} \sum_{k=1}^{|\phi_i|} x^i_{jh,k} = 1 \quad \forall i \in M,\ j \in J,\ h \in \Omega_j : \mu^h_j = i \quad (v^{RB}_1)$$

$$\sum_{j \in J} \sum_{h \in \Omega_j : \mu^h_j = i} x^i_{jh,k} \leq 1 \quad \forall i \in M,\ k = 1 \dots |\phi_i| \quad (v^{RB}_2)$$

$$f^i_k \leq C_{max} \quad \forall i \in M;\ k = 1 \dots |\phi_i| \quad (v^{RB}_3)$$

$$f^i_{k-1} + \sum_{j \in J} \sum_{h \in \Omega_j : \mu^h_j = i} p_{j,h} x^i_{jh,k} \leq f^i_k$$
$$\forall i \in M;\ k = 1 \dots |\phi_i| \quad (v^{RB}_4)$$

$$\sum_{i_1 : \mu^h_j = i_1} f^{i_1}_{k_1} - F(1 - x^{i_1}_{jh,k_1}) \leq$$
$$\sum_{i_2 : \mu^h_j = i_2} [f^{i_2}_{k_2} - p_{j,h+1} \cdot x^{i_2}_{j(h+1),k_2} + F(1 - x^{i_2}_{jh,k_2})]$$
$$\forall j \in J;\ h = 1 \dots n_j - 1$$
$$k_1 = 1 \dots |\phi_{\mu^h_j}|,\ k_2 = 1 \dots |\phi_{\mu^{h+1}_j}| \quad (v^{RB}_5)$$

Notiamo che, anche in questo caso, si ricercano solo le schedule *Earliest-start* tramite il fattore correttivo $\epsilon \sum_{j \in J} \sum_{k=1}^{|\phi_i|} f^i_k$.
Per quanto riguarda i vincoli descriviamo brevemente i primi quattro e dedichiamo invece all'ultimo una spiegazione più approfondita. v^{RB}_1 stabilisce l'assegnazione di ogni attività che richiede il generico macchinario M_i ad una sola posizione della sua sequenza, mentre v^{RB}_2 stabilisce che ogni posizione può ospitare al più un'attività. v^{RB}_3 è il classico vincolo sul Makespan e v^{RB}_4 è il vincolo di precedenza tra le attività che competono per lo stesso macchinario [$T_{i,j} \equiv T^h_j \in \phi_i$].

L'ultimo vincolo v^{RB}_5è più complesso e ha il compito di vincolare il job j a seguire la rotta prestabilita dall'istanza. Supponiamo per esempio che, per il generico job j, l'attività T^h_j sia collocata nella posizione k_1 del macchinario μ^h_j da essa richiesto (la variabile x relativa si attiva); ciò implica che l'attività $T_{j,h+1}$, qualsiasi sia la posizione k_2 occupata su μ^{h+1}_j, inizi non prima della fine di T^h_j ($f^{\mu^h_j}_{k_1} \leq s^{\mu^{h+1}_j}_{k_2}$).
Tutte le altre situazioni, per le quali ovviamente non è necessario definire una relazione di precedenza, sono trattate rendendo banale il vincolo tramite l'utilizzo del parametro F, che compare al primo membro con il segno $-$ e al secondo membro con il segno $+$.
Cerchiamo di comprendere come stimare il numero di vincoli per questa formulazione. Supponendo $n_j = m\ \forall j \in J$, ragioniamo indifferente su uno dei vincoli di v^{RB}_2, v^{RB}_3 o v^{RB}_4:

$$\sum_{i \in M} |\phi_i| \quad \text{numero effettivo vincoli}$$
$$|\phi_i| = n \quad \forall i \in M$$
$$\sum_{i \in M} |\phi_i| = mn$$

Procedendo con un ragionamento analogo per v_5^{RB} si ha che:

$$\sum_{j\in J}\sum_{h=1}^{n_j-1} |\phi_{\mu_j^h}||\phi_{\mu_j^{h+1}}| \quad \text{numero effettivo vincoli}$$
$$|\phi_i| = n \quad \forall i \in M$$
$$n_j = m \quad \forall j \in J$$
$$\sum_{j\in J}\sum_{h=1}^{n_j-1} |\phi_{\mu_j^h}||\phi_{\mu_j^{h+1}}| = n(m-1)n^2 \sim mn^3$$

Con ragionamenti analoghi a quelli visti si deducono le stime del numero di variabili coinvolte nella formulazione. Anche in questo caso concludiamo la trattazione riportando tabelle riassuntive relative al numero di vincoli e di variabili per la formulazione in esame.

	v_1^{RB}	v_2^{RB}	v_3^{RB}	v_4^{RB}	v_5^{RB}	TOT
NCONST	mn	mn	mn	mn	$\sim mn^3$	$O(mn^3)$

Tabella 5: Numero vincoli Rank Based Formulation (RB)

	$x^i_{jh,k}$	f^i_k	$C_{\max}$	TOT
NVAR	mn^2	nm	1	$O(mn^2)$

Tabella 6: Numero variabili Rank Based Formulation (RB)

3.4 Confronto tra le formulazioni

Si presenta ora ad un confronto delle formulazioni in termini prestazionali. I confronti sono stati realizzati su istanze generate randomicamente tramite AMPL, e risolte poi con il solver *Gurobi* nella sua versione *11.0.3*; il calcolatore usato è un *Lenovo* con processore *Intel(R) Core i5-12450H* di dodicesima generazione e *16 GB* di RAM installata.
Si sono testate le tre formulazioni su tre classi di istanze (ciascuna composta da trenta istanze) con $n = m = n_j = 5$ e tempi di processamento casuali (ma pur sempre interi) in diversi intervalli:

- Classe 1: $1 \leq p_{j,h} \leq 30$.
- Classe 2: $30 \leq p_{j,h} \leq 60$.
- Classe 3: $60 \leq p_{j,h} \leq 90$.

Tutte le grandezze di seguito analizzate sono espresse in termini di media per le varie classi di istanze e i tempi sono sempre misurati in secondi.
La scelta di queste classi è stata fatta per due motivi:

- Mostrare la dipendenza delle formulazioni TI(1) e TI(2) da $p_{j,h}$
- Esaminare istanze di piccole dimensioni risolubili all'ottimo in tempi ragionevolmente brevi.

Anzitutto, per facilitare la lettura dello scritto, riportiamo congiuntamente i risultati delle tabelle precedentemente esposte nella Tabella (7):

	NCONST	NVAR
DM	$O(2mn^2)$	$O(mn^2)$
TI(1)	$O(m(n+T))$	$O(mnT)$
RB	$O(nm^3)$	$O(mn^2)$

Tabella 7: Confronto formulazioni (quadro sinottico NVAR e NCONST)

3.4.1 TI(1)-TI(2)

Anzitutto confrontiamo le due diverse formulazioni Time Indexed proposte, definendo rispettivamente TI*(1) e TI*(2) le due formulazioni in cui l'Upper Bound T al valore ottimo del makespan viene fornito tramite l'eurstica Shifting Bottleneck.
Introdotta questa notazione, confrontiamo dunque le due formulazioni partendo dal numero di vincoli e dal numero di variabili.

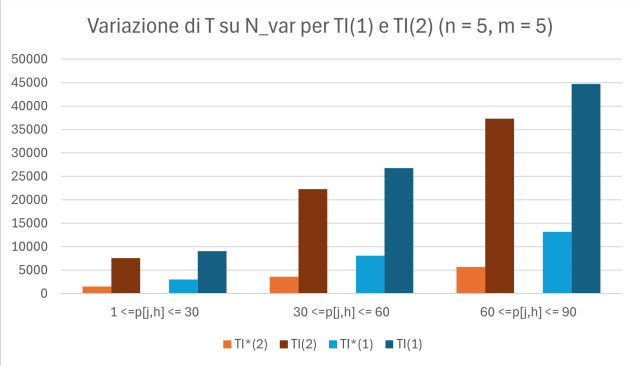


Figura 7: Effetto della variazione di T sul numero di variabili nelle Form TI

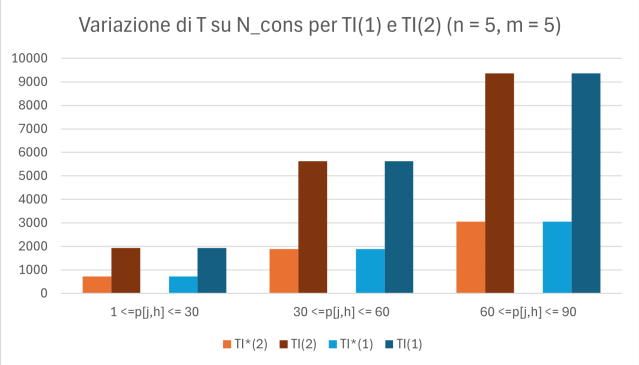


Figura 8: Effetto della variazione di T sul numero di vincoli nelle Form TI

Notiamo l'effetto benefico del fornire un buon UB iniziale per TI(1) e per TI(2), tanto nel numero di variabili quanto nel numero di vincoli. Risulta anche evidente la riduzione di NVAR passando da TI(1) (TI*(1)) a TI(2) (TI*(2)); per NCONST, invece, non si hanno differenze passando da una formulazione TI all'altra.
Risulta dunque evidente dalla sperimentazione quanto asserito nella descrizione del modello.
Queste discrepanze numeriche nel numero di vincoli e

variabili hanno come conseguenza un netto aumento del tempo di calcolo passando dalla Classe 1 alla Classe 3, come evidenziato dalla seguente figura:

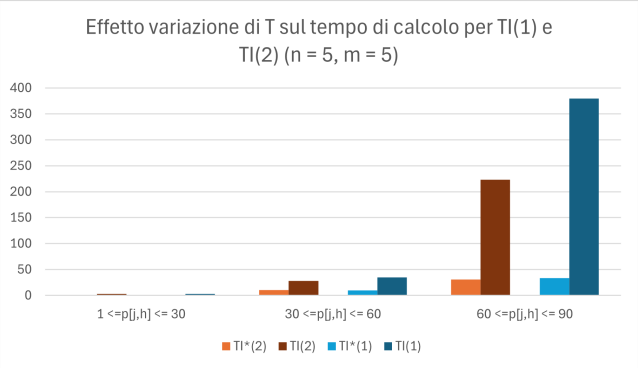


Figura 9: Effetto della variazione di T sul tempo di calcolo nelle Form TI

3.4.2 DM e RB

Per quanto riguarda le formulazioni DM e RB, esse hanno un numero di vincoli e variabili indipendente dall'istanza considerata; in ogni caso, l'istanza DM risulta essere più compatta in NVAR e NCONST come specificato dalla seguente figura:

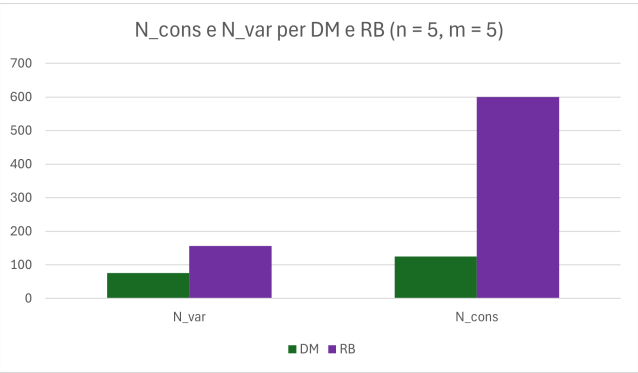


Figura 10: DM ed RB a confronto

Come vedremo, questo risulterà in un minor tempo di calcolo per DM rispetto a RB.

3.4.3 Tempi di calcolo a confronto

In via definitiva, dunque consideriamo la variazione dei tempi di calcolo tra le varie Classi di istanze per le formulazioni proposte. Per ovvi motivi, al fine di rappresentare le Formulazioni Time Indexed, si sono scelte TI*(1) e TI*(2):

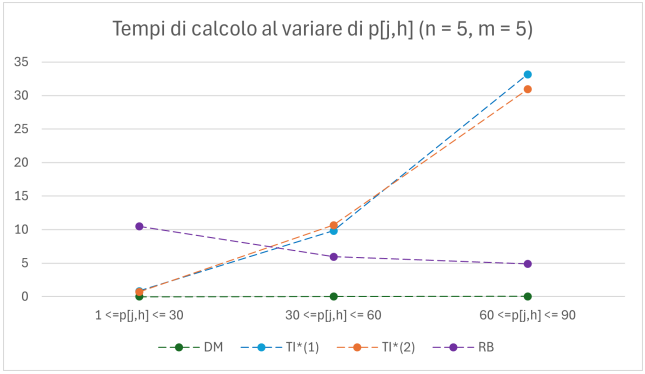


Figura 11: Tempi di calcolo tra le varie formulazioni

Le conclusioni che si traggono sono le suguenti:

1) A parità di n ed m, i tempi di calcolo delle formulazioni TI crescono con i $p_{j,h}$, mentre per DM ed RB restano circa costanti.
2) DM è migliore di RB.

Qualcuno potrebbe obiettare che un'analisi su istanze $n = 5$, $m = 5$ non è sufficiente per asserire che la formulazione DM sia la migliore (come invece si evince dall'analisi sperimentale proposta) e in un primo momento questo potrebbe sembrare sensato.
Riflettendo meglio però sul numero di vincoli e variabili dei problemi (Tabella 7) si può facilmente fornire una spiegazione teorica per questi risultati. Difatti anche se $p_{j,h} = 1$, $\forall\ T_j^h$, si avrebbe $T = nm$, rendendo NCONST[TI(1)] $\geq$ NCONST[RB] ed NVAR[TI(1)] $\geq$ NVAR[RB].
Possiamo dunque affermare che RB sia la migliore formulazione per $J_m||C_{max}$ (limitatamente alla risoluzione tramite *Gurobi* in *AMPL*), attestando che il Modello su grafo disgiuntivo è non solo intuitivo, ma anche efficiente in termini risolutivi.

4 Euristica Shifting Bottleneck (SBH): descrizione algoritmo

Faremo riferimento all'algoritmo descritto per la prima volta da (15, Adams et Al.), il quale rappresenta una delle euristiche migliori per il problema in esame. La procedura si basa sul modello del Grafo Disgiuntivo descritto in (2), per il quale una soluzione ammissibile, come detto, corrisponde ad un *orientamento aciclico* di $G = (V, E, A)$ (dove $A \triangleq \{A_1 \cup A_2\}$), ottenuto orientando gli archi disgiuntivi E.
Il makespan C_{max} della schedula associata a tale orientamento aciclico di G è pari alla lunghezza l_π del cammino (*critico*) di lunghezza massima π da s a t.
$J_m||C_{max}$ si riduce quindi a trovare l'ordinamento (aciclico) di G che minimizzi la lunghezza del cammino di lunghezza massima da s a t. In una rete **AON** si assume il peso dell'arco (i, j) pari al peso del nodo coda i:

$$d_{i,j} = p_i \quad \forall (i,j) \in E \cup A \tag{4}$$

Così formulato, il problema assume una struttura bilivello $min - max$, di difficile risoluzione; capiremo nel dettaglio come SBH superi tale ostacolo.

Prima di passare alla vera e propria proceudura algoritmica

sarà utile introdurre due Lower Bound per il problema sia per avere una stima (qualora non si conosca l'ottimo) della qualità della soluzione, sia per comprendere la filosofia dell'algoritmo.

4.1 Lower Bound

Per introdurre i primi due Lower Bound, è necessario richiamare alcune nozioni relative al Project Scheduling con regole di precedenza sulle attività. In generale, dato un problema di Scheduling con vincoli di precedenza $FS_{ij}^{min}(0)$ ($f_i \leq s_j$), risorse illimitate e obiettivo minimizzazione del Makespan ($R_\infty|prec|C_{max}$), il valore ottimo C^*_{max} può essere trovato associando una rete **AON** aciclica in cui si aggiungono eventualmente una sorgente (pozzo) qualora non sia presente un'attività priva di predecessori (successori) e calcolando su di essa il costo del cammino più lungo dal pozzo alla sorgente. Tale cammino prende il nome di *Cammino Critico*.
Il concetto di cammino critico, è naturalmente collegato ai concetti di *Earliest Start Time* (EST) e di *Latest Finish Time* (EFT) delle attività della rete. Definiamo questi concetti come segue:

- EFT_i (dove i è la generica attività-nodo della rete in esame): minimo tempo entro cui un'attività può cominciare. Notiamo che $EST_t = Cmax^*$.
- LFT_i (dove i è la generica attività-nodo della rete in esame): tempo massimo entro cui un'attività può terminare, pena aumento del Makespan.

Queste definizioni, tutt'altro che operative, possono essere specificate nel caso delle reti **AON** che stiamo esaminando come segue. Per quanto riguarda EST_i, esso rappresenta di fatto il *ready time* r_i dell'attività i; considerando i vincoli $FS_{ij}^{min}(0)$ entranti in Figura (12), il calcolo di $EST_i(\triangleq r_i)$ viene strutturato come:

$$\begin{aligned} &min \quad r_i \\ &s.t \quad r_i \geq r_h + p_h \quad \forall h \in Pred(i) \\ &\text{ovvero} \\ &r_i = \max_{h \in Pred(i)} \{r_h + p_h\} \end{aligned} \tag{5}$$

Nel caso di LFT_i, esso rappresenta la *due date* d_i dell'attività i (un ritardo rispetto a d_i causa la penalità di aumento del Makespan); considerando in questo caso i vincoli $FS_{ij}^{min}(0)$ uscenti in Figura (13), il calcolo di $LFT_i(\triangleq d_i)$ viene strutturato come:

$$\begin{aligned} &max \quad d_i \\ &s.t \quad d_i \leq d_j - p_j \quad \forall j \in Succ(i) \\ &\text{ovvero} \\ &d_i = \min_{h \in Pred(i)} \{d_j - p_j\} \end{aligned} \tag{6}$$

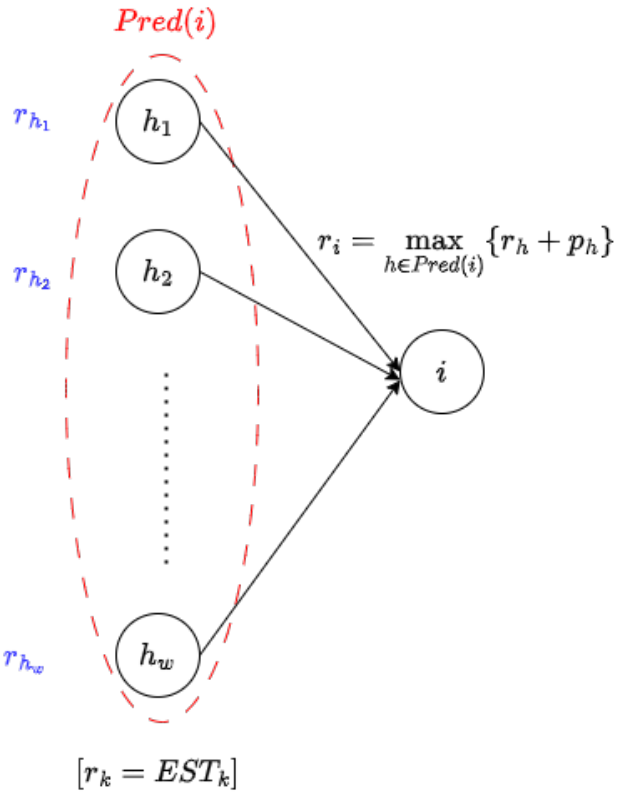


Figura 12: Calcolo EST

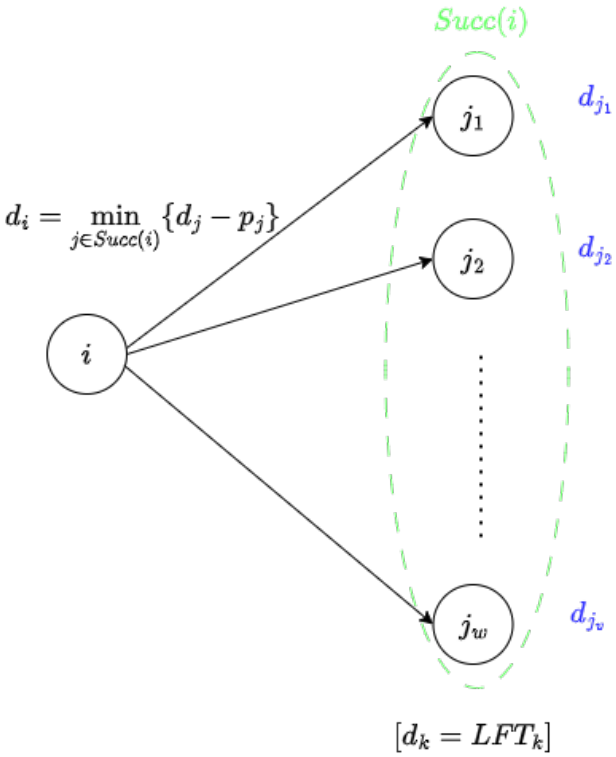


Figura 13: Calcolo LFT

Tali definizioni, unite al fatto che la rete sia aciclica, consentono di applicare le procedure algoritmiche *Forward* e *Backward* per il calcolo rispettivamente di r_i e d_i; supponendo di aver orinato topologicamente le q attività (nodi) e indicando con $T_{(k)}$ l'attività che occupa la posizione k dell'ordinamento topologico (ovviamente $T_{(1)} = s,\ T_{(q)} = t$), si eseguono:

Algoritmo 1 Procedura Forward

$r_{(1)} \leftarrow 0$
for $k = 2$ to q **do**
  $r_{(k)} \leftarrow \max\{r_{(j)} + p_{(j)} \mid T_{(j)} \in \text{Pred}(T_{(k)})\}$
end for

Algoritmo 2 Procedura Backward

$d_{(q)} \leftarrow r_{(q)}$
for $k = q - 1$ downto 0 **do**
  $d_{(k)} \leftarrow \min\{d_{(j)} - p_{(j)} \mid T_{(j)} \in \text{Succ}(T_{(k)})\}$
end for

Le procedure sono da eseguire sequenzialmente ed hanno ciascuna complessità di $O(q + |A \cup E|)$.

Ultimo concetto da introdurre è quello di *Total Float* di un'attività i (TF_i), definito come il massimo ritardo possibile sul suo inizio (completamento), senza alterare il Makespan. Sulla base delle definizioni appena fornite:

$$\begin{aligned} TF_i &= d_i - (r_i + p_i) = LFT_i - EFT_i \\ EFT_i &= \text{ tempo minimo di fine di un'attivita} \end{aligned} \tag{7}$$

Un'attività per cui $TF_i = 0$ ($\Leftrightarrow d_i = p_i + r_i$) viene definita $critica$; le attività che fanno parte del percorso critico sono tutte critiche. Infatti dal momento che l'attività i non può iniziare prima di r_i e deve essere processata per p_i, il margine di slittamento possibile rispetto a r_i ovvero a d_i è nullo.

Consideriamo per una generica istanza $J_m||C_{max}$ il grafo disgiuntivo G depurato dagli archi congiuntivi ($E = \emptyset$); questa situazione corrisponde ad aver rilassato *completamente* i vincoli sulle Risorse e il relativo valore del Makespan ottimo (r_t) è pertanto un primo LB per il problema originario; indichiamo questo valore con LB_1.
Risulta opportuno cercare di migliorare tale Lower Bound, rilassando solo *parzialmente* i vincoli sulle risorse, ovvero supponendo che tutte le macchine tranne una possano eseguire più di un'attività alla volta. Questa operazione equivale a reinserire soltanto la clique di archi disgiuntivi relativa al macchinario $i \in M$ non rilassato.
L'orientamento di tali archi disgiuntivi causerà un incremento del Makespan pari a δ_i, dove δ_i è il minimo tempo addizionale necessario per sequenziare le attività che competono per il macchinario i.
Dalle considerazioni precedenti risulta che l'attività $T_{i,j}$ non può iniziare prima di $r_{i,j}$, mentre il tempo addizionale richiesto è pari alla sua *Tardiness* rispetto a $d_{i,j}$. Poichè ogni ritardo rispetto a $d_{i,j}$ causa un aumento del Makespan di pari entità, il tempo addizionale δ_i è pari alla massima tardiness delle attività $T_{i,j}$.
Calcolare δ_i equivale dunque a risolvere il problema $1|r_j|L_{max}$ sulla macchina i e sulle attività $T_{i,j}$ che la richiedono, con date di rilascio pari a $r_{i,j}$ e date di scadenza pari a $d_{i,j}$.
In definitiva:

$$\begin{aligned} &\delta_i = max(0, L^*_{max}) \quad \forall i \in M \\ &LB_2 \triangleq LB_1 + \max_i\{\delta_i\} \\ &i^* = \arg\max_i\{\delta_i\} \end{aligned} \tag{8}$$

Definiamo i^*, la macchina *Bottleneck* di questa particolare configurazione del sistema. Il concetto di individuazione della macchina bottleneck, oltre ad essere il cuore delle procedura SBH, fa riferimento ad un'importante approccio industriale noto come *Theory of Constraints* (TOC); esso si basa su due importanti assunti:

1) Ogni sistema reale deve essere caratterizzato da almeno un vincolo.
2) L'esistenza di vincoli rappresenta opportunità di miglioramento.

In particolare, il processo di miglioramento ciclico proposto dalla TOC, molto simile alla ruota di Deming, ricalca in parte la SBH; per riferimenti più generali alla *Theory of Constraints* si rimanda a (21, Rahman).
Computazionalmente calcolare LB_2 equivale a risolvere m istanze di $1|r_j|L_{max}$, un problema *NP-hard* in senso forte ma comunque risolubile in tempi ragionevoli con algoritmi di Branch and Bound (per opportuni riferimenti si rimanda a (14)).
Ci si potrebbe chiedere se la risoluzione all'ottimo di questi problemi sia effettivamente necessaria, ma, riflettendo bene sul concetto di Lower Bound, ci si convincerebbe presto che questo è inevitabile.
Considerando infatti il caso banale con un solo macchinario, calcolare LB_2 equivale a calcolare una soluzione ammissibile per il problema. Qualora però $1|r_j|L_{max}$ non fosse risolto all'ottimo, la soluzione derivante dal relativo orientamento degli archi congiuntivi presenterebbe Makespan strettamente maggiore di C^*_{max}, negando la definzione stessa di Lower Bound.

4.2 Un Lower Bound Alternativo

In ultimo vogliamo presentare un Lower Bound per $J_m||C_{max}$, che non ha alcuna relazione con i primi due e che pertanto viene aggiunto per completezza. Abbiamo visto nella Sezione (3.1), che un grafo disgiuntivo è composto da Clique e Catene di attività non contemporaneamente eseguibili.
Nella migliore delle ipotesi ciascuno di questi insiemi di attività permane nel sistema per un tempo pari alla sua durata complessiva; pertanto si definisce:

$$\begin{aligned} &LB_3 \triangleq max(\max_{i \in M} \sum_j p_{i,j},\ \max_{j \in J} \sum_i p_{i,j}) \\ &LB_2 \geq LB_1 \geq \max_{j \in J} \sum_i p_{i,j} \end{aligned} \tag{9}$$

Dunque non è possibile stabilire a priori una relazione tra LB_2 ed LB_3.

4.3 La procedura

La SBH è una procedura iterativa che consta di m iterazioni articolate come nella seguente figura:

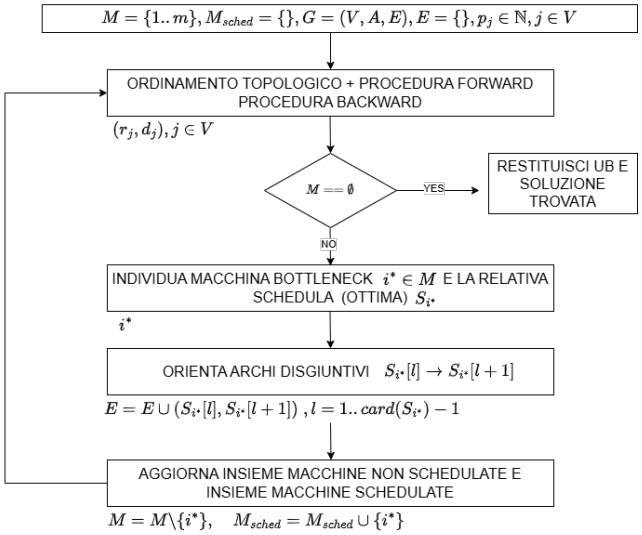


Figura 14: Architettura della SBH

Si disinguono due differenti insiemi:

- M: insieme delle macchine da schedulare.
- M_{sched}: insieme delle macchine schedulate (per le quali sono stati orientati aciclicamente i relativi archi congiuntivi).

Si considera inizialmente il grafo disgiuntivo G depurato dagli archi congiuntivi ($E = \emptyset$), ovvero si considera ogni macchina ancora da schedulare (topologia di rete analoga a quella di LB_1). Alla generica iterazione k, tra le macchine ancora non schedulate, si individua la macchina bottleneck i^*, come fatto per il calcolo di LB_2 e si orientano i relativi archi disgiuntivi secondo la schedula ottima di $1|r_j|L_{max}$; il processo continua fintanto che non si è trovato un orientamento aciclico di E, ovvero una soluzione ammissibile per $J_m||C_{max}$

L'algoritmo codificato in (15) prevede inoltre una procedura di ottimizzazione locale alla fine di ogni iterazione, in modo da analizzare gli effetti congiunti delle scelte localmente ottime rispetto alla sola macchina bottleneck. Definendo E_k gli archi disgiuntivi orientati per il generico macchinario $k \in M_{sched}$, si ha:

Algoritmo 3 Ricerca Locale in SBH

for k in M_{sched} **do**
  $E \leftarrow E \backslash E_k$
  do: Topological Sort, Forward, Backward
  Solve $1|r_j|L_{max}$ upon k
  $E_k \leftarrow \{S_k[l], S_k[l+1],\ l = 1 \dots card(S_k) - 1\}$
end for

Illustreremo ora con due esempi il motivo della necessità di questa procedura, che migliora ovviamente la qualità della soluzione trovata mettendo in discussione le scelte fatte in iterazioni precedenti. Questo è vero anche quando scelte indefferenti dal punto di vista locale portano a soluzioni ammissibili di valore diverso; situazioni di indifferenza (da gestire tramite arbitrari criteri *Break-Even*) possono nascere in due casi:

1) BE_1: Si hanno più schedule ottime sulla macchina Bottleneck.
2) BE_2: Esistono due o più macchinari candidati ad $\arg\max_{i \in M}\{\delta_i\}$.

Gli esempi a cui si è sopra accennato riguardano proprio questi due casi.

4.3.1 *Situazioni di indifferenza in SBH*

Esaminiamo separatamente le due situazioni di indifferenza con esempi pratici, risolti tramite **AMPL** e graficati tramite **MATLAB**.

-Primo caso: BE_1

Si consideri l'istanza di $J_m||C_{max}$ con $n = m = 4$ riportata di seguito. Le operazioni sono etichettate con $T_{i,j}$ dove, come di consueto, i indica il macchinario richiesto e j il job di appartenenza. Le rotte dei singoli job sono invece definite dalle catene.

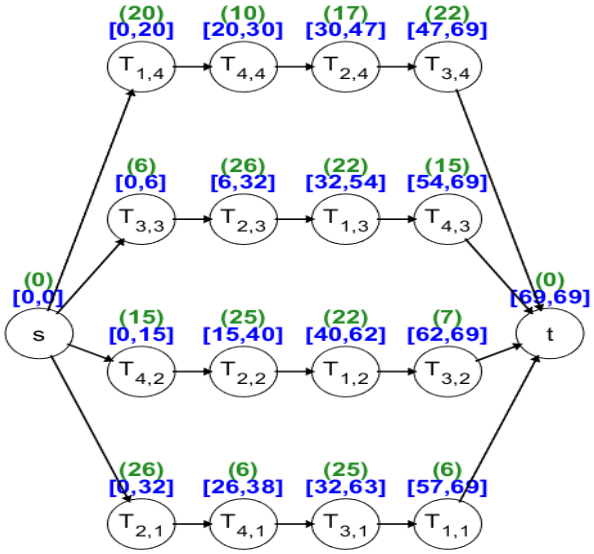


Figura 15: situazione iniziale istanza BE_1

Nella figura la tripla di numeri sovrastanti ogni attività è così definita:

- $[r_{i,j}, d_{i,j}]$: valori trovati tramite le procedure Forward e Backward.
- $(p_{i,j})$: tempo di processamento dell'attività $T_{i,j}$.

Alla prima iterazione i vari δ_i sono:

i	1	2	3	4
δ_i	14	47	17	0

Tabella 8: (BE_1) δ_i alla prima iterazione

Questo implica ovviamente che la macchina bottleneck è

la macchina 2. Per motivi che dipendono dall'istanza, il problema $1|r_j|L_{max}$ presenta almeno due soluzioni ottime equivalenti S_2^1 ed S_2^2:

S_1^1	1	3	2	4
S_2^2	1	2	3	4

Tabella 9: (BE_2) soluzioni equivalenti per la prima macchina bottleneck

Oss: la scelta dipende dal valore del seed di lavoro per il solver *Gurobi*, utilizzato anche in questo caso per la risoluzione di $1|r_j|L_{max}$ nelle varie iterazioni della Shifting Bottleneck. Si utilizza in un caso il seed 42 e in un altro caso il seed 33; Se si fosse invece utilizzato un algoritmo di Branch and Bound costruito ad hoc per $1|r_j|L_{max}$, si sarebbe potuta scegliere (sempre in modo arbitrario) la sequenza ottima utilizzare.

Riportiamo nella Figura (22) le due diverse configurazioni di rete ottenute scegliendo S_2^1 o S_2^2. Da queste due differenti strutture di E_2, l'algoritmo evolve producendo due distinte soluzioni ammissibili, una di valore 123 e una di valore 116 (ottima in quanto $r_t = 116 = LB_1$ alla prima iterazione).
Riassumiamo i due grafi disgiuntivi orientati corrispondenti nella Figura (23).
Infine, riportiamo qui di seguito le schedule corrispondenti alle due soluzioni trovate.



Figura 16: (BE_1) Schedula trovata con S_2^1

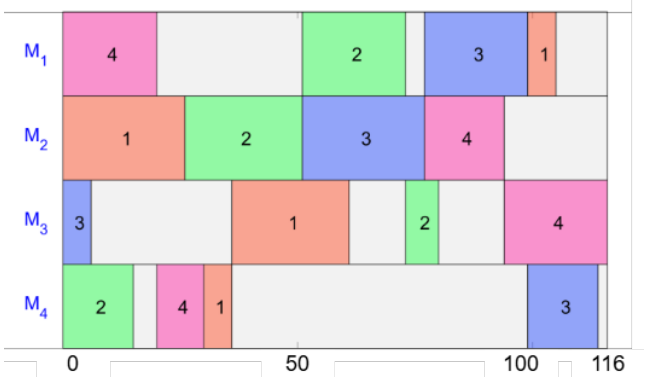


Figura 17: (BE_1) Schedula trovata con S_2^2

-Secondo caso: BE_2
Si consideri l'istanza di $J_m||C_{max}$ con $n = 3$, $m = 4$ riportata di seguito. La rappresentazione ha lo stesso significato di quella precedentemente introdotta.

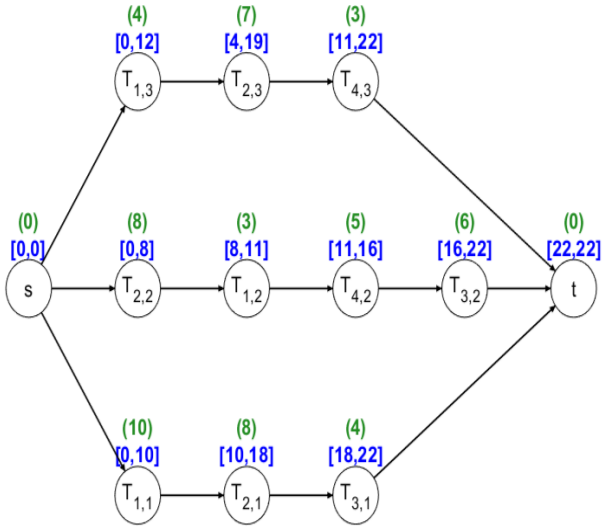


Figura 18: situazione iniziale istanza BE_2

Alla prima iterazione i vari δ_i sono:

i	1	2	3	4
δ_i	5	5	4	0

Tabella 10: (BE_2) δ_i alla prima iterazione

Questo implica una situazione di indifferenza rispetto alla scelta della macchina Bottleneck tra M_1 ed M_2; prescindendo questa scelta dall'utilizzo di seed diversi nel solver *Gurobi*, il seed in questo caso viene bloccato a 42.

Riportiamo in Figura (24) le due diverse configurazioni di rete ottenute scegliendo come prima macchina bottleneck rispettivamente M_1 ed M_2. L'algoritmo, anche in questo caso, produce due diverse soluzioni ammissibili, una di valore 28 (di cui si può dimostrare l'ottimalità) e un'altra di valore 32.
Anche in questo caso riportiamo le due schedule trovate in modo da evidenziare le differenze:

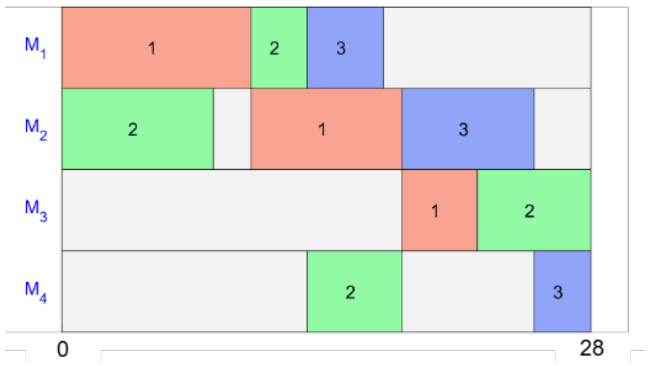


Figura 19: (BE_2) Schedula trovata con M_1 come prima macchina bottleneck

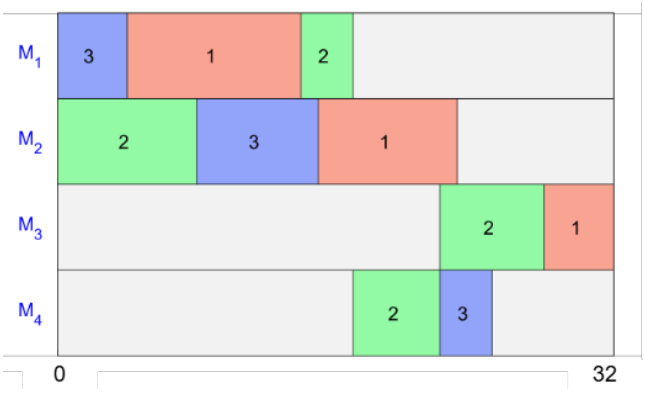


Figura 20: (BE_2) Schedula trovata con M_2 come prima macchina bottleneck

Abbiamo quindi compreso il motivo della scelta di inserire la procedura (4) alla fine di ogni iterazione con due esempi pratici.
Bisogna fare una considerazione ulteriore: in generale, la scelta di orientare gli archi della macchina bottleneck non ha garanzia di essere la scelta ottima dal punto di vista del problema nel suo complesso. Questo è infatti solo un criterio euristico, che non valuta l'effettivo valore di "lungo periodo" della scelta effettuata.
In compenso si ottiene un algoritmo che, come vedremo, riesce a fornire buoni UB al valore ottimo in tempi molto rapidi, anche senza l'inserimento della ottimizzazione locale sopra discussa.

4.4 Analisi di complessità

Risulta a questo punto necessario discutere brevemente della complessità computazionale del metodo proposto. L'algoritmo schematizzato in Fiugura (14), alla generica iterazione k risolve $m - k + 1$ sottoproblemi su macchina singola; è proprio il complesso di tali problemi a costituire il reale peso computazionale della procedura.
Come detto, $1|r_j|L_{max}$ è un problema *NP-HARD* e la sua risoluzione all'ottimo tramite metodi di Branch and Bound, non consente una stima a priori della complessità. Qualitativamente, però, al crescere del numero di attività che incidono sullo stesso macchinario, la complessità cresce.
In particolare, in totale si risolvono:

$$\sum_{i=1}^{m} i = \frac{m(m+1)}{2}$$

sottoproblemi di ottimizzazione. Nella Sezione (4.6.2), esamineremo cosa succede fissando il numero di operazioni ma variando contestualmente m ed n, per comprendere cosa sia computazionalmente conveniente per la procedura SBH.

A questo vanno aggiunte le $3m$ procedure di Ordinamento Topologico, Forward e Backward (una per ogni iterazione) e le m procedure di orientamento degli archi disgiuntivi, tutte ovviamente polinomiali nel numero di archi della rete.

4.5 Strutture Dati

In questo caso, si è scelto di utilizzare la matrice di Adiacenza (attività-attività) A della rete di progetto; si assegna al nodo s l'ID 1 e si procede sulle diverse catene dei job (assegnando ID crescenti) per arrivare ad assegnare l'ID q al nodo t; come esempio, forniamo di seguito la matrice A inizializzata dell'istanza in Figura (15):

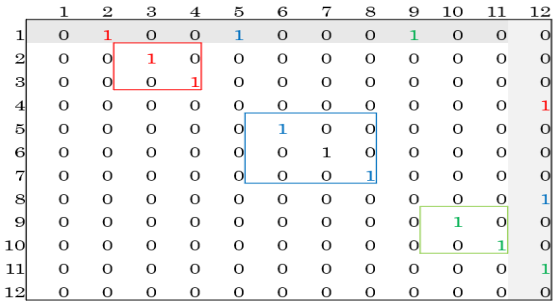


Figura 21: Matrice di Adiacenza alla prima iterazione istanza BE_1

In particolare notiamo i blocchi di matrici identità evidenziati in colore diverso; tali blocchi rappresentano gli archi esistenti tra le operazioni dei job in diretta precedenza tra loro (archi della catena). La dimensione del blocco identità è ovviamente inferiore di un'unità alla dimensione della catena. Per ciascun job sono poi evidenziati con lo stesso colore un 1 sulla prima riga (arco da s alla prima attività del job) e un 1 sulla ultima colonna (arco dall'ultima attività del job a t). Questa matrice viene opportunamente popolata di 1^i sulla base del sequenziamento della macchina bottleneck ad ogni iterazione.

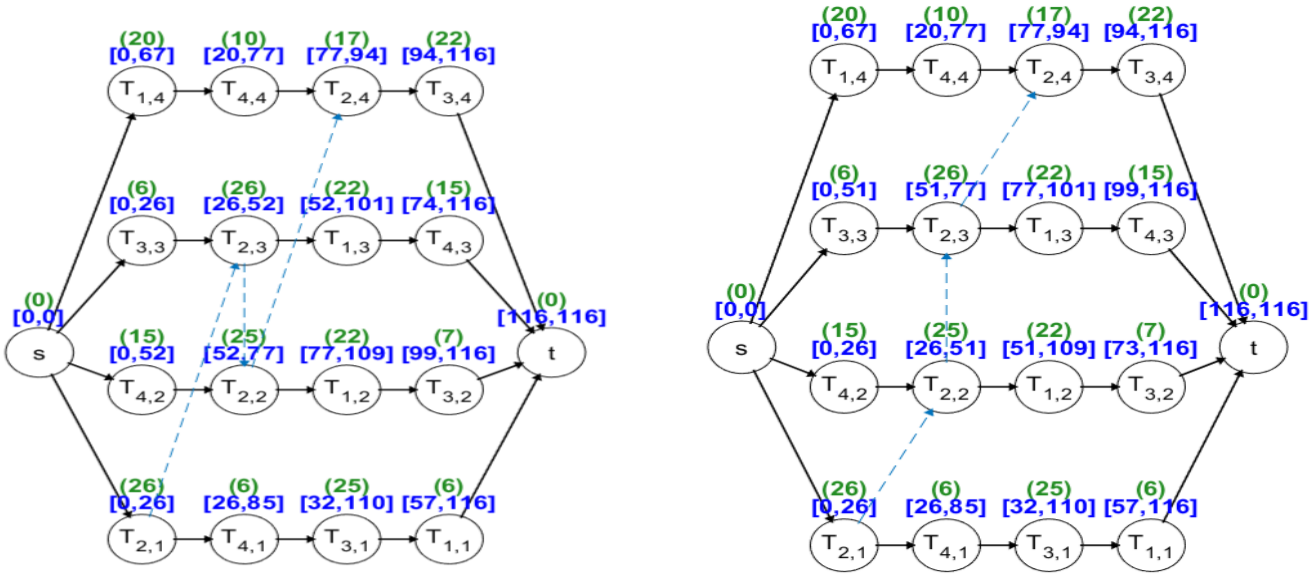
Per quanto riguarda invece la risoluzione dei vari sottoproblemi $1|r_j|L_{max}$, l'associazione delle attività ai vari macchinari viene realizzata con una matrice di Incidenza Macchine-Attività B definita come segue:

$$B_{i,j} = \begin{cases} 1 \text{ se attivita' } j \text{ richiede macchina } i \\ 0 \quad \text{altrimenti} \end{cases} \tag{10}$$

Indicando con q il numero di attività, queste strutture matriciali sono algoritmicamente desumibile dalla matrice P (Sezione (1.1)) con la seguente procedura (in cui viene anche associato alle varie attività il tempo di processamento):

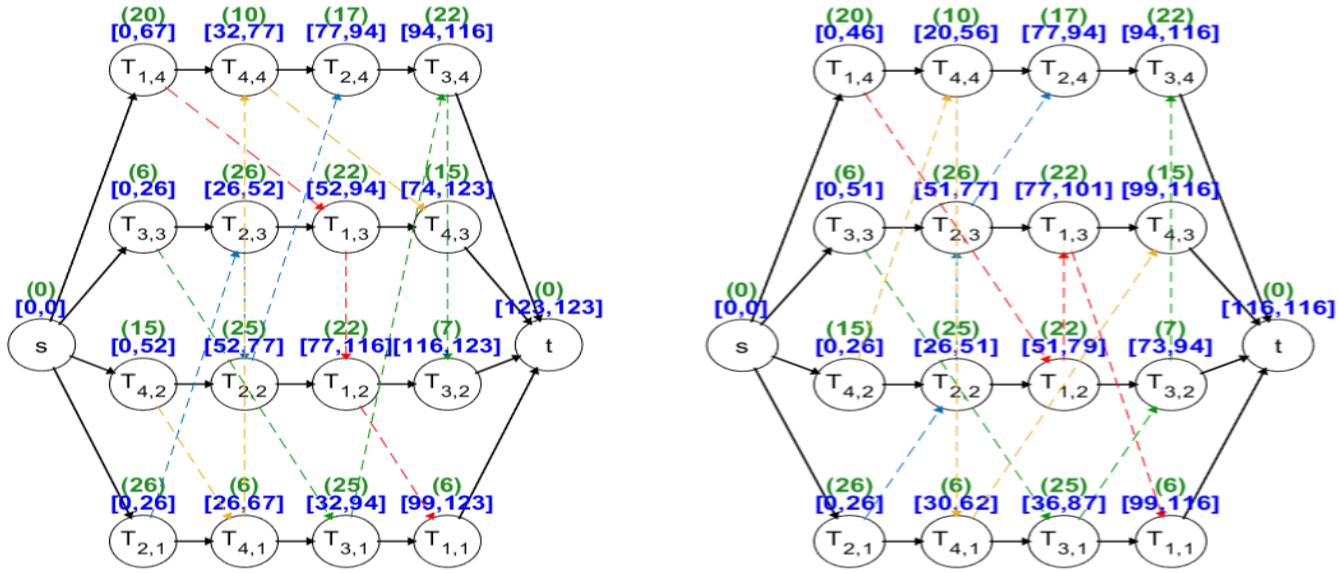
Algoritmo 4 Costruisci Disgiuntive Graph

$last = 2$
$A \in \{0,1\}^{qxq}$
$A_{i,j} \leftarrow 0 \quad i = 1 \dots q, = 1 \dots q$
for $j = 1$ to n **do**
  for $h = 1$ to $max_j\{n_j\}$ **do**
    if $P_{j,2h} \geq 0$ **then**
      $A_{j,succ} \leftarrow 1$
      $indx = P_{j,2h-1}$
      $B_{indx,succ} \leftarrow 1$
      $p_{succ} = P_{j,2h}$
      $succ \leftarrow succ + 1$
    end if
  end for
end for



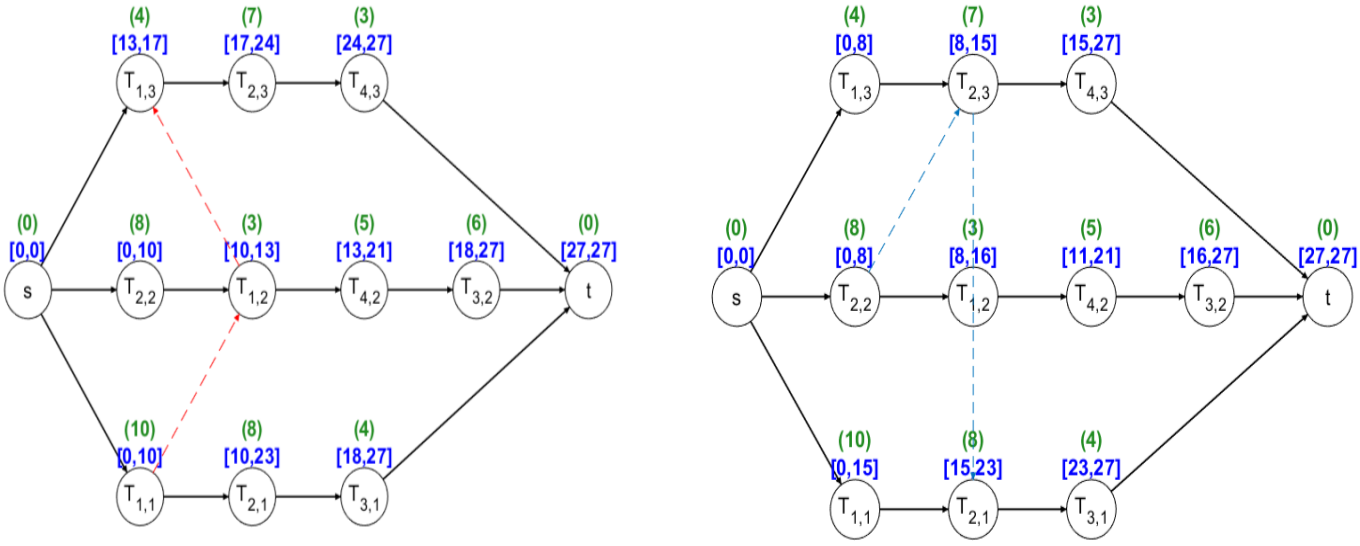
(a) S_2^1 (b) S_2^2

Figura 22: (BE_1) Configurazione di rete alla prima iterazione: confronto tra scelte diverse



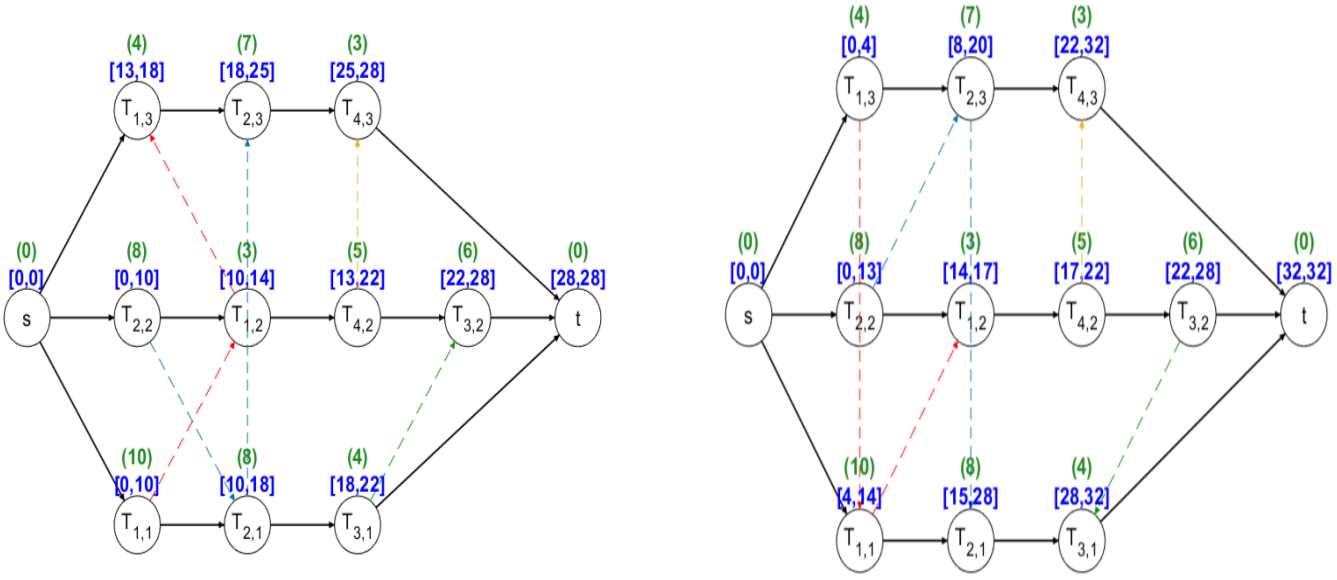
(a) Configurazione finale da S_2^1 (b) Configurazione finale da S_2^2

Figura 23: (BE_1) Configurazione di rete finale: confronto tra scelte diverse



(a) M_1 (b) M_2

Figura 24: (BE_2) Configurazione di rete alla prima iterazione: confronto tra scelte diverse



(a) Configurazione finale da M_1 come prima macchina Bottleneck

(b) Configurazione finale da M_2 come prima macchina Bottleneck

Figura 25: (BE_2) Configurazione di rete finale: confronto tra scelte diverse

4.6 Risultati Sperimentali

Veniamo ora alla parte implementativa della procedura proposta e ai risultati sperimentali, relativi a due aspetti differenti :

1) Confronto con risoluzione all'ottimo (tempi di calcolo e valori) e qualità dei vari Lower Bound, per istanze di dimensione crescente.
2) Trade off tra numero job e numero macchinari a parità di attività (nodi) della rete.

In entrambi i casi le istanze sono state generate casualmente.

4.6.1 Confronto con ottimo e Lower Bound

Si sono eseguiti confronti su 3 classi di istanze quadrate ($n = m = n_j$), mettendo a confronto non solo il tempo medio di calcolo ma anche l'errore (in termini relativi) dell'UB fornito dalla Shifting Bottleneck rispetto all'ottimo e ai Lower Bound.
Per la risoluzione all'ottimo si è utilizzata la formulazione su Grafo Disgiuntivo, che abbiamo mostrato essere la migliore nella relativa sperimentazione; il solutore utilizzato anche in questo caso è $Gurobi$.
Le classi generate, contenenti ciascuna 30 istanze, presentano tempi di processamento interi uniformemente distribuiti in $[1, 30]$. Le classi sono le seguenti:

1) Classe 1: $n = m = n_j = 7$.
2) Classe 2: $n = m = n_j = 10$.
3) Classe 3: $n = m = n_j = 13$.

Compariamo anzitutto i tempi di calcolo tramite la seguente tabella e il relativo grafico:

Dim	OPT(DM)_TIME	SBH_TIME	p-value
7x7	0.10	1.65	4.30×10^{-33}
10x10	2.20	5.77	1.89×10^{-12}
13x13	105.51	18.26	2.49×10^{-39}

Tabella 11: Confronto tabellare tempi di calcolo: OPT(DM) vs. SBH

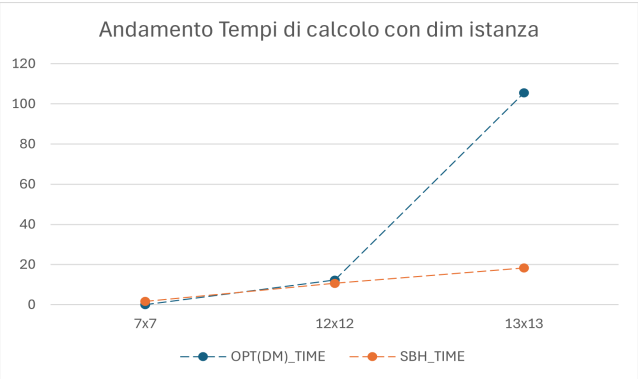


Figura 26: Confronto grafico tempi di calcolo: OPT(DM) vs. SBH

Notiamo la presenza di un *p-value*, il quale è relativo al *test-t ad una coda* di confronto tra le medie di campioni eteroschedastici (test di *Welch*). Notiamo che per le istanze 7x7 e quelle 10x10, la risoluzione iterativa degli $m(m+1)/2$ problemi non risulta conveniente;
Già per istanze di medie dimensioni (13x13), il vantaggio computazionale della SBH è più che evidente (si noti il valore del p-value prossimo a 0).

A questo punto è lecito chiedersi quanto sia costata questa riduzione del tempo di calcolo in termini di qualità della soluzione generata. Compariamo pertanto l'errore relativo medio (su ciascuna delle classi da 30 istanze) della soluzione ottima trovata rispetto all'ottimo e ai Lower Bound LB_2 ed LB_3 nella Tabella 12.

Dim	OPT (%)	LB2 (%)	LB3 (%)
7x7	11.0%	20.7%	35.2%
10x10	20.5%	29.5%	46.1%
13x13	27.4%	37.4%	56.3%

Tabella 12: Confronto tabellare errore relativo SBH: errore vero e sovrastima rispetto a LB

Per *errore relativo* di una soluzione di valore z rispetto ad un target t si intende:

$$\epsilon_r = \frac{z - t}{t} \tag{11}$$

Notiamo anzitutto anzitutto che LB_2 è qualitativamente migliore di LB_3 nonostante presentino entrambi un trend crescente all'aumentare della dimensione delle istanze. L'andamento crescente di entrambi gli errori relativi è qualitativamente spiegato sulla base della definizione di tali Lower Bound:

1) LB_2 viene definito sul costo del cammino critico alla prima iterazione. Risulta ovvio che un aumento del numero di macchinari e del numero di job (dunque del numero di operazioni) rende tale sottostima peggiore rispetto al valore del cammino critico all'ultima iterazione; per convincersene, si noti che il numero di archi disgiuntivi inseriti tra la seconda e la tredicesima iterazione nelle istanze 13x13 è pari a $12 \cdot 12 = 144$, mentre nelle istanze 7x7 solo di $6 \cdot 6 = 36$.
2) LB_3 viene definito sulla base del massimo *workload* necessario per processare le attività non sequenziabili in parallelo. Considerando anche in questo caso la necessità di rispettare i numerosi vincoli di precedenza dati da $A \cup E$ all'ultima iterazione, risulta sensato il peggioramento di questa sottostima con il crescere delle istanze.

Il medesimo trend crescente si riscontra sull'errore relativo rispetto al valore ottimo.
Di seguito rappresentiamo l'andamento degli errori relativi delle varie classi di istanze, al fine di evidenziare quanto detto.

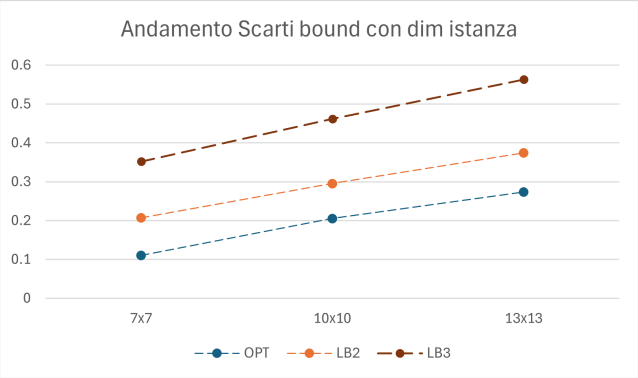


Figura 27: Confronto grafico errori relativi: OPT(DM) vs. LB

Riportiamo nell'Appendice A i seguenti grafici per le varie classi di istanze proposte:

1) tempi di calcolo (in secondi) delle singole istanze sia per la risoluzione con DM che per quella con SBH. Si noti la minore varianza dei tempi di calcolo relativi alla procedura SBH, rispetto a quelli relativi alla risoluzione ottima.
2) Distribuzioni degli errori relativi e grafici a dispersione dei valori assoluti. Si noti lo spostamento degli istogrammi verso destra (errori concentrati in intervalli crescenti) e i maggiori scostamenti (riscalati in maniera opportuna) dei valori assoluti con l'aumentare delle dimensioni dell'istanza

Oss: in entrambi i casi sono stati eliminati gli eventuali valori *outlier*, la cui causa potrebbe essere ricondotta a interferenze nella procedura di calcolo, ovvero alla risoluzione di istanze troppo (o troppo poco) complesse rispetto al caso medio.

4.6.2 Analisi variazione (n,m) a parità di numero di attività

In questo caso abbiamo esaminato 150 istanze (con tempi di processamento interi uniformemente distribuiti in $[30, 90]$) da 225 attività, dimensionalmente rilevanti dal punto di vista applicativo.
Le istanze sono raggruppate in 5 classi (ciascuna da 30 istanze) definite in base a diversi valori della coppia (n, m):

1) Classe 1: $n = 5,\ m = 45$.
2) Classe 2: $n = 25,\ m = 9$.
3) Classe 3: $n = 15,\ m = 15$.
4) Classe 4: $n = 25,\ m = 9$.
5) Classe 5: $n = 45,\ m = 5$.

Oss: per semplicità, ma senza perdita di generalità, si considerano istanze con con job composti da m attività; definiamo queste istanze come *regolari*.
Il motivo di questa ultima sperimentazione risiede nella volontà di analizzare il *trade-off computazionale* tra numero di sottoproblemi da risolvere (legato a m) e complessità degli stessi (legata a n). Per un'istanza *regolare* con n job ed m è necessario risolvere $\frac{m(m+1)}{2}$ sottoproblemi ciascuno definito su n attività.
Si hanno dunque due gradi di libertà che agiscono contestualmente:

- n aumenta, m diminuisce: diminuisce il numero di sottoproblemi ma aumenta la complessità elementare.
- n diminuisce, m aumenta: aumenta il numero di sottoproblemi, la diminuisce la complessità elementare.

Poichè i sottoproblemi sono risolti tramite *Branch and Bound*, un'analisi teorica risulta praticamente impossibile; pertanto facciamo affidamento alla sperimentazione, cercando di interpretare i risultati ottenuti.
Oss: le istanze qui presentate non sono state risolte all'ottimo viste le eccessive dimensioni dei problemi in esame; si confronterà pertanto il valore dell'UB fornito da SBH con i LB discussi.
Riassumiamo dunque i tempi di calcolo (in secondi), gli errori relativi (in media sulle classi di istanze) e il numero di sottoproblemi risolti tramite la seguente tabella: Iniziamo

Dim: nxm	**LB2 (%)**	**LB3 (%)**	**SBH_TIME**	N_SOTTOPROB
5x45	3.55 %	4.09 %	60.45	1035
25x9	18.50 %	21.26 %	34.61	325
15x5	43.36 %	72.32 %	34.96	120
25x9	26.26 %	26.95 %	37.55	45
45x5	9.48 %	9.48 %	36.80	15

Tabella 13: Confronto tabellare istanze quadrate e rettangolari

questa volta con un'analisi dell'andamento degli scarti rispetto ai Lower Bound. Notiamo come le istanze quadrate siano nettamente sfavorite dal punto di vista della qualità delle soluzioni ottenute; questo fatto, però, potrebbe anche essere dovuto ad un peggioramento della qualità dei Lower Bound.

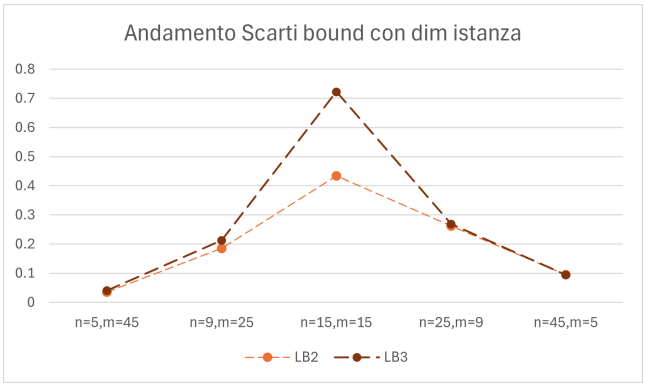


Figura 28: Confronto grafico errori relativi SBH: istanze quadrate e rettangolari

Sarebbe interessante eseguire ulteriori sperimentazioni, al fine di comprendere se il peggioramento dell'errore relativo è da attribuirsi al peggioramento del Lower Bound, dell'Upper Bound ovvero al combinarsi delle due cause.
Un'ipotetica causa dell'andamento di LB2 potrebbe essere relativa al peso delle iterazioni della SBH sulla lunghezza del cammino critico:

1) n elevato: dopo la prima iterazione devono essere schedulati pochi macchinari per giungere ad una

configurazione ammissibile, tutti con un workload elevato.

2) m elevato: dopo la prima iterazione devono essere schedulati molti macchinari per giungere ad una configurazione ammissibile, tutti con un workload molto basso.

Questi due effetti sembrano controbilanciarsi agli estremi; tendendo invece alle istanze quadrate questo effetto benefico sembrerebbe invece svanire.
Si noti inoltre una grande vicinanza della qualità di LB2 e LB3 bound per le istanze estreme $(n = 5, m = 45)$ e $(n = 45, m = 5)$.

Per quanto riguarda il tempo di calcolo, l'analisi sperimentale ha confermato che la complessità computazionale, fissato il numero di attività, è dominata dal numero di macchinari.

Per convincersi di questo fatto si analizzino contestualmente il grafico del tempo medio di calcolo e del numero di sottoproblemi.

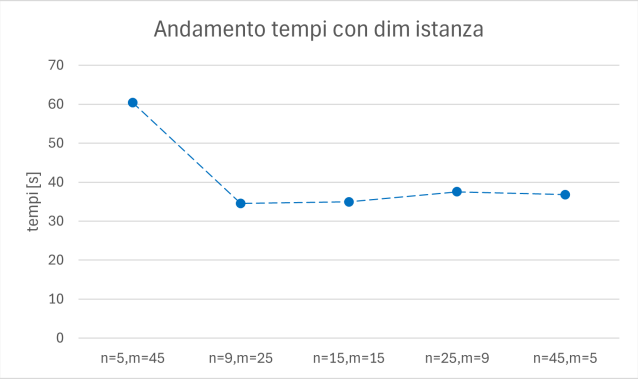


Figura 29: Confronto grafico tempi di calcolo SBH: istanze quadrate e rettangolari

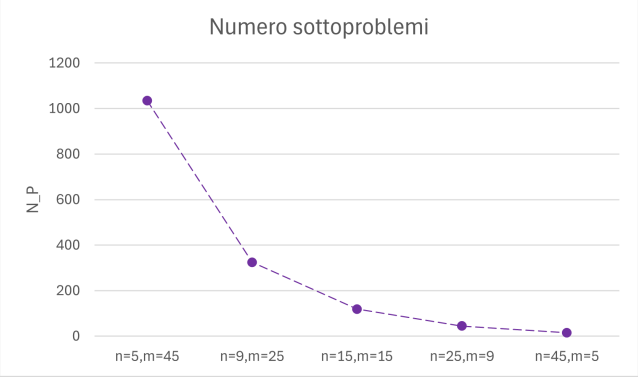


Figura 30: Confronto numero sottoproblemi SBH: istanze quadrate e rettangolari

Le due curve ottenute sono molto simili tra loro, e attestano che l'aumento di complessità elementare di $1|r_j|L_{max}$, sia più che compensato dalla riduzione di m. Questo fatto, nel caso della sperimentazione condotta, è da attribuire alla grande efficienza di *Gurobi*, nonchè ovviamente alla drastica (quadratica) riduzione del numero di sottoproblemi per valori decrescenti di m.
In Appendice B sono riportati i tempi medi di calcolo per la risoluzione di $1|r_j|L_{max}$ nelle varie iterazioni di SBH, per le classi di istanze analizzate.

5 Conclusioni

Mediante le varie riflessioni teoriche e le relative analisi sperimentali presentate, abbiamo avuto modo di provare nel dettaglio la grande complessità del problema $J_m||C_{max}$. Si ricordi, che tale problema spesso presenta semplificazioni eccessive rispetto ai casi reali, richiedendo ulteriori specifiche o modifiche strutturali (vincoli e funzioni di prestazione); tali aggiunte rendono queste generalizzazioni ancora più complesse e sfidanti.
A ben vedere, dunque, i problemi di Scheduling costituiranno ancora per molto uno degli aspetti più delicati della Ricerca Operativa (e della Computer Science in generale), rendendo sempre più necessari approcci multidisciplinari integrati.

Giunti dunque alla conclusione del presente documento, cerchiamo di trarre qualche conclusione, riassumendo i risultati ottenuti per aree tematiche:

- **Formulazioni matematiche**
  1) La formulazione DM presenta prestazioni migliori rispetto a TI e RB (Sezione 3.4).
  2) La formulazione TI ha una complessità risolutiva strettamente dipendente dall'istanza fornita. Con un leggero abuso teorico, si pensi a questo modello come un modello *pseudopolinomiale* (concetto definito per i soli algoritmi ma che si presta molto bene a descrivere questa caratteristica della formulazione TI) (Sezione 3.4).
- **SBH**;
  1) La SBH necessita di procedure di ottimizzazione locale che mettano in discussione le decisioni di breve periodo prese sulla base della macchina bottleneck (Sezione 4).
  2) La qualità della soluzione di istanze quadrate peggiora con l'aumentare della dimensione dell'istanza (Sezione 4.6.1).
  3) Già per istanze con $n = m = 13$ la SBH presenta tempi di calcolo inferiori di circa l'80% rispetto alla risoluzione con DM, con soluzioni di qualità tutto sommato accettabile. Possiamo affermare che la SBH è un algoritmo euristico costruttivo efficiente ed efficace (Sezione 4.6.1).
  4) A parità di numero di operazioni, il tempo di calcolo resta circa costante per valori di m non troppo elevati; per valori di m molto elevati il tempo di calcolo subisce un brusco aumento. Resta aperta la discussione relativa al confronto qualitativo per i motivi discussi nella relativa Sezione 4.6.2.

RIFERIMENTI BIBLIOGRAFICI

[1] S. Giordani, *Dispense del corso MGSC*, 2024-2025.

[2] R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. R. Kan, "Optimization and approximation in deterministic sequencing and scheduling: a survey," in *Annals of discrete mathematics*. Elsevier, 1979, vol. 5, pp. 287–326.

[3] H. Xiong, S. Shi, D. Ren, and J. Hu, "A survey of job shop scheduling problem: The types and models," *Computers & Operations Research*, vol. 142, p. 105731, 2022.

[4] R. L. Sisson, "Methods of sequencing in job shops—a review," *Operations Research*, vol. 7, no. 1, pp. 10–29, 1959.

[5] M. R. Garey, D. S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Mathematics of operations research*, vol. 1, no. 2, pp. 117–129, 1976.

[6] B. Giffler and G. L. Thompson, "Algorithms for solving production-scheduling problems," *Operations research*, vol. 8, no. 4, pp. 487–503, 1960.

[7] J. Carlier and É. Pinson, "An algorithm for solving the job-shop problem," *Management science*, vol. 35, no. 2, pp. 164–176, 1989.

[8] J. R. Barker and G. B. McMahon, "Scheduling the general job-shop," *Management Science*, vol. 31, no. 5, pp. 594–598, 1985.

[9] E. Balas, "Machine sequencing via disjunctive graphs: an implicit enumeration algorithm," *Operations research*, vol. 17, no. 6, pp. 941–957, 1969.

[10] B. Roy and B. Sussmann, "Les problemes d'ordonnancement avec contraintes disjonctives," *Note ds*, vol. 9, 1964.

[11] A. A. B. Pritsker, L. J. Waiters, and P. M. Wolfe, "Multiproject scheduling with limited resources: A zero-one programming approach," *Management science*, vol. 16, no. 1, pp. 93–108, 1969.

[12] H. M. Wagner, "An integer linear-programming model for machine scheduling," *Naval research logistics quarterly*, vol. 6, no. 2, pp. 131–140, 1959.

[13] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied mathematics*, vol. 10, no. 1, pp. 196–210, 1962.

[14] E. L. Lawler, J. K. Lenstra, A. H. R. Kan, and D. B. Shmoys, "Sequencing and scheduling: Algorithms and complexity," *Handbooks in operations research and management science*, vol. 4, pp. 445–522, 1993.

[15] J. Adams, E. Balas, and D. Zawack, "The shifting bottleneck procedure for job shop scheduling," *Management science*, vol. 34, no. 3, pp. 391–401, 1988.

[16] H. Matsuo, "A controlled search simulated annealing method for the general jobshop scheduling problem," *Working paper 03-04-88, University of Texas at Austin*, 1988.

[17] P. J. Van Laarhoven, E. H. Aarts, and J. K. Lenstra, "Job shop scheduling by simulated annealing," *Operations research*, vol. 40, no. 1, pp. 113–125, 1992.

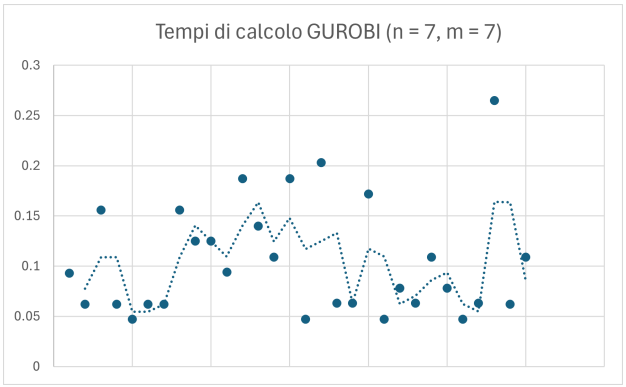
[18] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and X. Chi, "Learning to dispatch for job shop scheduling via deep reinforcement learning," *Advances in neural information processing systems*, vol. 33, pp. 1621–1632, 2020.

[19] E. D. Taillard, "Parallel taboo search techniques for the job shop scheduling problem," *ORSA journal on Computing*, vol. 6, no. 2, pp. 108–117, 1994.

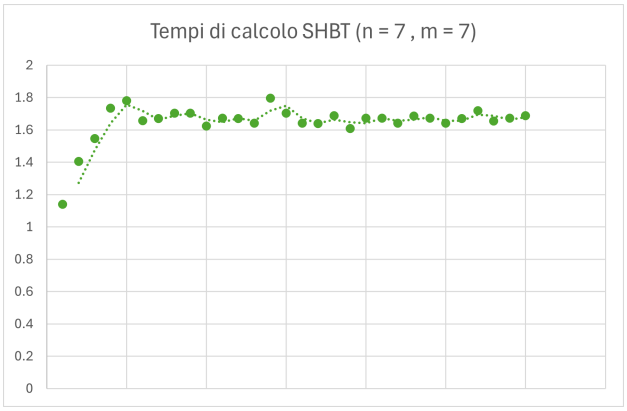
[20] C. Zhang, P. Li, Z. Guan, and Y. Rao, "A tabu search algorithm with a new neighborhood structure for the job shop scheduling problem," *Computers & Operations Research*, vol. 34, no. 11, pp. 3229–3242, 2007.

[21] S.-u. Rahman, "Theory of constraints: a review of the philosophy and its applications," *International journal of operations & production management*, vol. 18, no. 4, pp. 336–355, 1998.

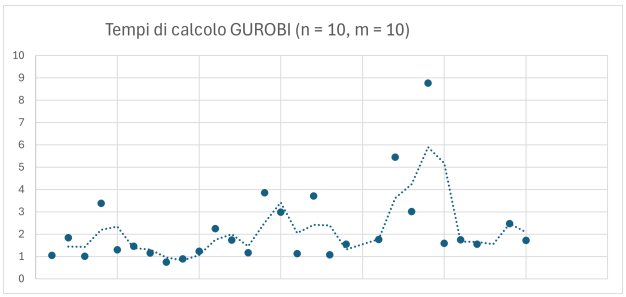
APPENDICE A
GRAFICI RELATIVI ALLA SEZIONE (4.6.1): CONFRONTO CON OTTIMO E LOWER BOUND



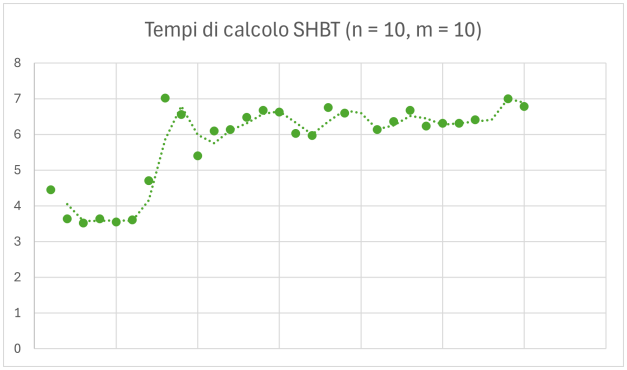
(a) Tempi di calcolo [s] relativi alla formulazione DM per $m = n = 7$



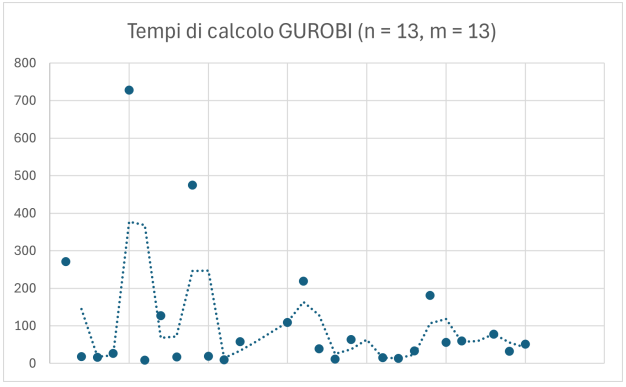
(b) Tempi di calcolo [s] relativi alla SBH per $m = n = 7$



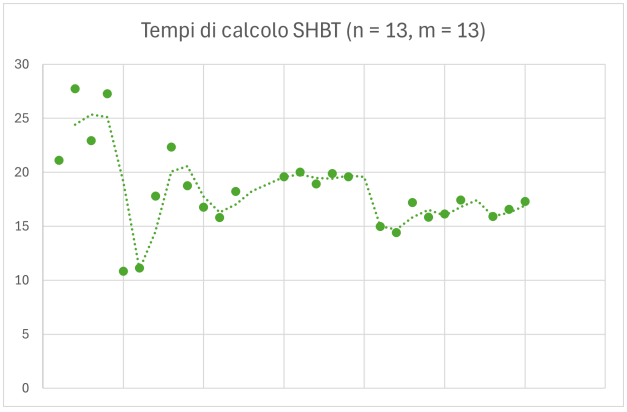
(c) Tempi di calcolo [s] relativi alla formulazione DM per $m = n = 10$



(d) Tempi di calcolo [s] relativi alla SBH per $m = n = 10$

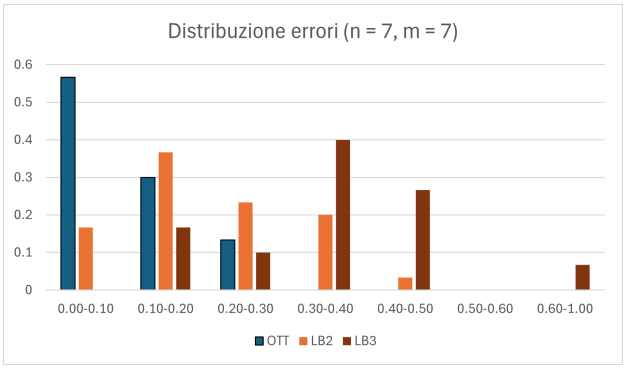


(e) Tempi di calcolo [s] relativi alla formulazione DM per $m = n = 13$

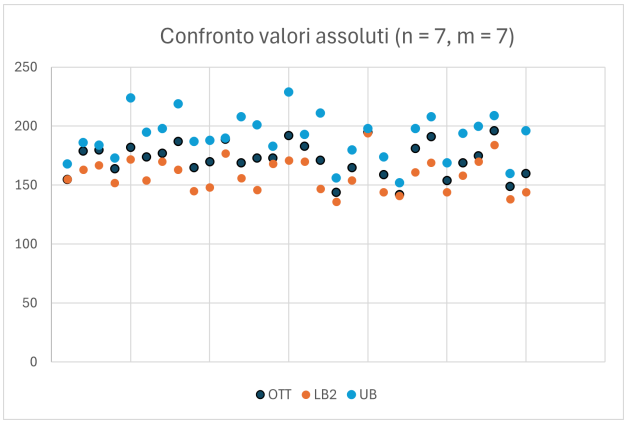


(f) Tempi di calcolo [s] relativi alla SBH per $m = n = 13$

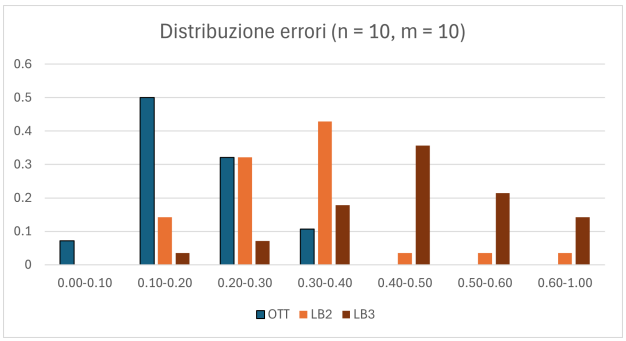
Figura 31: Tempi di calcolo sperimentazione SBH, confronto con DM (Sezione 4.6.1)



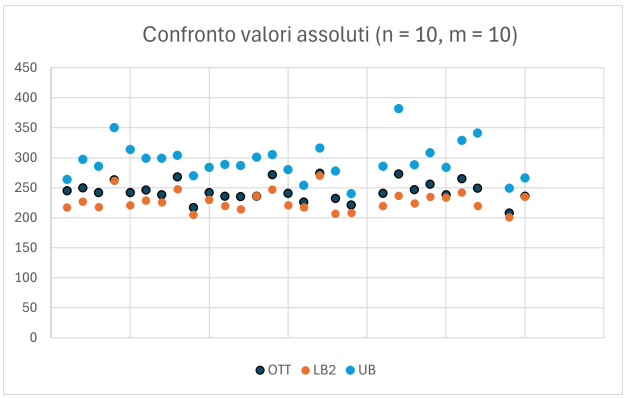
(a) Distribuzione errori relativi per $m = n = 7$



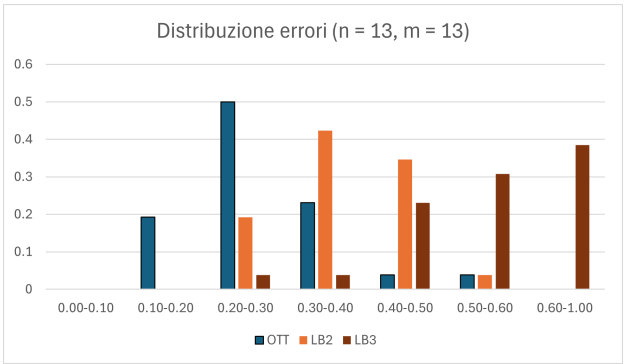
(b) Dispersione valori assoluti per $m = n = 7$



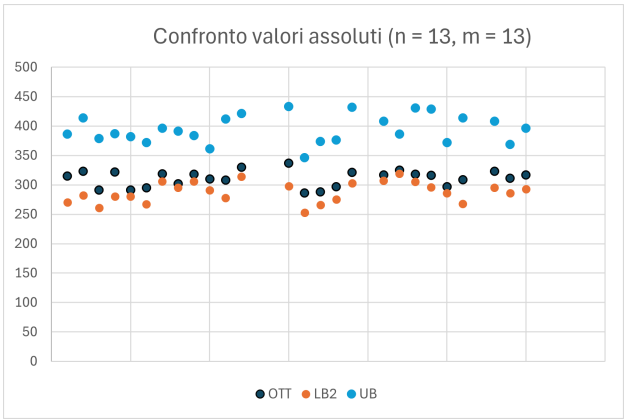
(c) Distribuzione errori relativi per $m = n = 10$



(d) Dispersione valori assoluti per $m = n = 10$



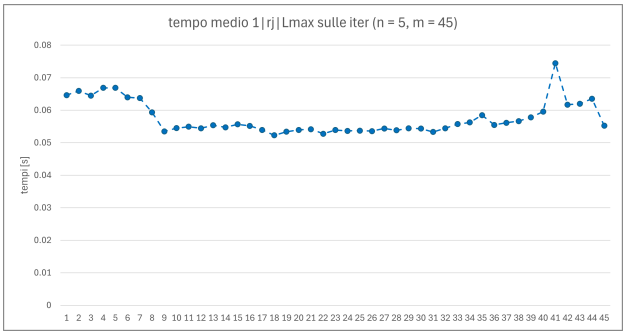
(e) Distribuzione errori relativi per $m = n = 13$



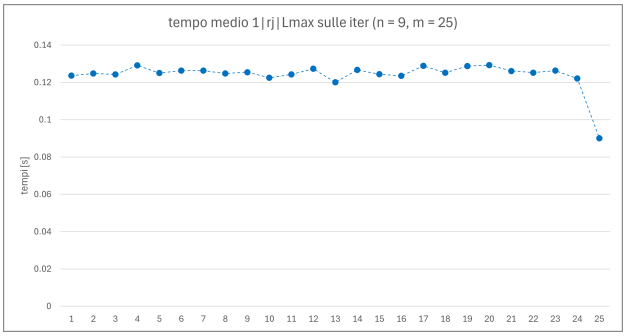
(f) Dispersione valori assoluti per $m = n = 12$

Figura 32: Errori relativi e valori assoluti sperimentazione SBH (Sezione 4.6.1)

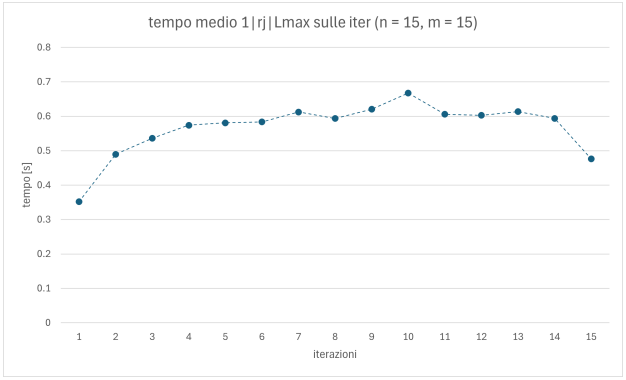
APPENDICE B
TEMPI MEDI DI CALCOLO $1|r_j|L_{max}$



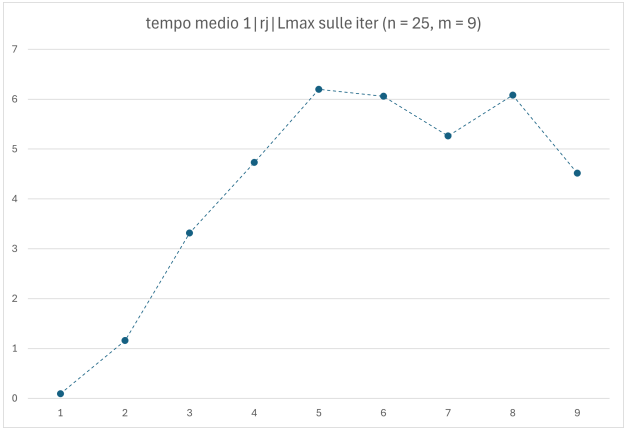
(a) Tempi medi di calcolo $1|r_j|L_{max}$, (n = 5, m = 45)



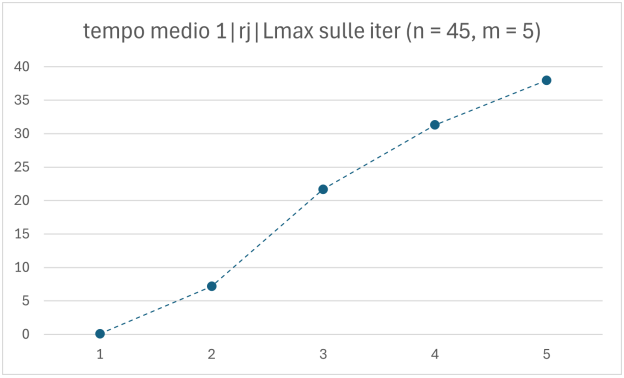
(b) Tempi medi di calcolo $1|r_j|L_{max}$, (n = 9, m = 25)



(c) Tempi medi di calcolo $1|r_j|L_{max}$, (n = 15, m = 15)



(d) Tempi medi di calcolo $1|r_j|L_{max}$, (n = 25, m = 9)



(e) Tempi medi di calcolo $1|r_j|L_{max}$, (n = 45, m = 5)

Figura 33: Tempo medio di calcolo sottoproblemi SBH per le varie iterazioni